

PROYECTO FINAL SERCAMPONET

Segundo de Desarrollo de Aplicaciones Web

Laura Olmedilla García
Asier Martínez Muñoz

2026

HOJA 0

Enlace a github:

https://github.com/AsierMartinez2000/Proyecto_Sercampo_BaseDatos.git

Enlace a la web donde se ha desplegado la aplicación:

<https://proyecto-sercamponet.vercel.app/>

Enlace a github del pdf de la memoria:

https://github.com/AsierMartinez2000/Proyecto_Sercampo_BaseDatos/blob/develop/Anexos/Memoria/Memoria_Final_SercampoNET_AsierMartinez_LauraOlmedilla.pdf

Enlace a github del pdf del manual de usuario de la aplicación:

https://github.com/AsierMartinez2000/Proyecto_Sercampo_BaseDatos/blob/develop/Anexos/Manual%20de%20Usuario/Manual%20de%20Usuario%20-%20SercampoNET.pdf

Usuarios creados para el uso de la aplicación:

1. Usuario Admin
 - a. Usuario: Laura
 - b. Contraseña: 1234
2. Usuario Estándar
 - a. Usuario: Asier
 - b. Contraseña: 1234
3. Usuario Conductor:
 - a. Usuario: Conductor
 - b. Contraseña: 1234

Enlace a github de los scripts para crear e inicializar la base de datos:

https://github.com/AsierMartinez2000/Proyecto_Sercampo_BaseDatos/blob/develop/backend/mysql/init.sql

Tabla de contenido

1. Descripción del proyecto.....	5
1.1. Características generales.....	5
1.2. Objetivos y alcance.....	7
2. Análisis del sector / mercado.....	8
2.1. Prospectiva del Título en el sector.....	8
2.2. Evolución y tendencia del sector.....	9
2.3. Normativa y documentación técnica específica.....	9
3. Plan de ejecución.....	10
3.1. Diagrama/cronograma de flujo de procesos (Diagrama de Gantt).....	10
3.2. Proceso de desarrollo software.....	11
3.2.1. Fase de análisis.....	11
3.2.1.1. Tipos de usuario.....	12
3.2.1.2. Descripción de requisitos.....	12
3.2.1.3. Casos de Uso.....	14
3.2.1.4. Guía de estilo.....	34
3.2.1.5. Prototipo del sitio web.....	38
3.2.1.6. Mapa de navegación.....	38
3.2.2. Fase de desarrollo.....	39
3.2.2.1. Base de datos.....	39
3.2.2.1.1. Análisis de requisitos de datos de la aplicación.....	39
a. Identificación de entidades e interrelaciones entre entidades.....	39
b. identificación de atributos de cada entidad.....	40
3.2.2.1.2 Diseño lógico de datos.....	43
3.2.2.1.3 Paso del modelo lógico (E/R) al modelo relacional (tablas).....	44
a. Identificación de atributos de cada tabla, sus tipos y tamaños....	44
b. Identificación de claves principales y claves candidatas.....	47
c. Identificación de las reglas de integridad y seguridad de la base de datos.....	48
d. Modelo relacional (tablas).....	49
3.2.2.1.4 Aplicación de reglas de normalización al modelo relacional...	50
3.2.2.1.5 Tipos de datos para el sistema gestor seleccionado.....	52
3.2.2.1.6 Scripts de creación de tablas e inserciones iniciales.....	53
3.2.2.2. Servidor.....	54
3.2.2.2.1. Lista de funciones en php.....	54
3.2.2.3. Cliente.....	57
3.2.2.3.1. Diseño de la interfaz.....	58
3.2.2.3.2. Accesibilidad.....	63
3.2.2.3.3. Usabilidad.....	64

3.2.2.3.4. Desarrollo web entorno cliente.....	66
a. Formularios y su validación.....	66
b. Manejo y gestión de eventos: Teclado, ratón y estados de la ventana.....	69
c. Gestión y almacenamiento de datos e información en el cliente.	71
d. Modificación del DOM.....	72
e. Animaciones, efectos, cambios dinámicos de estilos etc... para dinamizar la parte visible al cliente.....	72
f. Comunicación AJAX.....	74
g. Comunicación asíncrona con el servidor.....	76
3.2.3. Fase de despliegue.....	76
3.2.3.1. Despliegue utilizando un hosting.....	76
3.2.3.2. Despliegue en un equipo local propio.....	81
3.3. Seguimiento y control de incidencias.....	82
3.4. Indicadores de calidad de procesos.....	84
4. Recursos materiales.....	84
4.1. Inventario, valorado de medios.....	88
4.2. Presupuesto económico.....	89
5. Recursos humanos.....	90
5.1. Organización.....	90
5.2. Contratación.....	92
5.3. Prevención de riesgos laborales.....	92
6. Viabilidad técnica.....	94
6.1. Estudio de viabilidad técnica.....	94
7. Viabilidad económica-financiera.....	95
7.1. Inversiones y gastos.....	95
7.2. Financiación.....	97
7.3. Viabilidad económico-financiera.....	97
8. Conclusión.....	98
Bibliografía/Webgrafía.....	100
ANEXOS.....	100

1. Descripción del proyecto

El proyecto tiene como finalidad el desarrollo de una aplicación web dinámica orientada a la gestión integral de una empresa dedicada a la recogida y reciclaje de aceite, mediante contenedores distribuidos en diferentes ubicaciones geográficas. La solución propuesta, integra tecnologías tanto del lado del cliente como del servidor, así como el diseño y explotación de una base de datos relacional.

La aplicación permitirá centralizar la información relativa a clientes, contenedores, rutas de recogida, conductores, productos entregados y registros de actividad, proporcionando una herramienta que facilite la toma de decisiones, la trazabilidad de las operaciones y la optimización de los recursos logísticos.

El sistema está diseñado bajo un enfoque modular y escalable, permitiendo la incorporación de nuevas funcionalidades en fases posteriores, como la geolocalización avanzada o sistemas de alertas automatizadas.

Este proyecto surge por la necesidad en la empresa Sercampo, de una digitalización adecuada y funcional de la empresa, realizada de manera manual diariamente desde hace 12 años, pero que actualmente limita su capacidad de expansión. Tras una breve presentación de su problema y entrevistarnos con Gustavo como supervisor y con Elena, la persona encargada de dicho registro de información, se desarrolló la idea de la aplicación que se ha desarrollado durante este periodo de prácticas.

1.1. Características generales

La aplicación desarrollada presenta una arquitectura cliente-servidor y ofrece una interfaz web accesible mediante autenticación de usuarios. Las principales características funcionales son las siguientes:

Gestión de usuarios

- Sistema de autenticación mediante inicio de sesión.
- Gestión de perfiles de usuario con diferentes roles.
- Edición de datos personales y cierre de sesión.

Gestión de clientes

- Registro de nuevos clientes en el sistema.
- Consulta de clientes mediante distintos criterios (nombre, CIF, municipio).
- Modificación de la información existente.

Gestión de contenedores

- Consulta de contenedores asociados a clientes.
- Almacenamiento de información técnica (tipo, capacidad, mercancía, estado).
- Registro de coordenadas geográficas (latitud y longitud).
- Control de periodos de recogida.

Gestión de productos

- Administración de los productos que da la empresa a los clientes.
- Asociación de productos a recogidas realizadas.

Gestión de recogidas

- Registro de datos de recogida (fecha, litros recogidos, incidencias, estado).
- Asociación de recogidas a rutas y contenedores.
- Control de visitas realizadas por los conductores.

Gestión de rutas y logística

- Definición y planificación de rutas.
- Asignación de conductores y vehículos a rutas específicas.
- Relación entre rutas y contenedores a visitar.
- Consulta de actividad diaria de los conductores.

Gestión de conductores, vehículos y zonas

- Administración de conductores.
- Control de vehículos (ITV, mantenimiento, seguros).
- Organización territorial mediante zonas y municipios.

Panel de administración

- Interfaz centralizada para la gestión de entidades principales:
 - Clientes
 - Productos
 - Conductores
 - Zonas
- Control de acceso según rol del usuario.

Funcionalidades adicionales y futuras

- Sistema de alertas periódicas para la recogida de contenedores.
- Visualización geográfica de contenedores mediante mapas.
- Indicadores visuales de capacidad de los contenedores.
- Consulta detallada de rutas realizadas por conductor en fechas concretas.
- Mejora del sistema de autenticación (seguridad avanzada).

1.2. Objetivos y alcance

Desarrollar una aplicación web completa, funcional y escalable que permita gestionar de forma eficiente las operaciones de una empresa de recogida de residuos, integrando la gestión de clientes, contenedores, rutas y recogidas en un único sistema centralizado.

Diseñar e implementar una base de datos relacional normalizada que garantice la integridad y coherencia de los datos.

Desarrollar una interfaz web intuitiva y accesible para los distintos tipos de usuario.

Implementar operaciones CRUD (Crear, Leer, Actualizar y Eliminar) para las principales entidades del sistema.

Gestionar la autenticación y autorización de usuarios.

Facilitar la consulta y explotación de datos mediante filtros y búsquedas avanzadas.

Establecer una base sólida que permita futuras ampliaciones funcionales.

El alcance del proyecto comprende el desarrollo completo de una aplicación web funcional orientada a la gestión operativa de una empresa dedicada a la recogida y reciclaje de aceite. La solución engloba tanto el diseño como la implementación de los distintos componentes necesarios para su correcto funcionamiento, incluyendo la definición de la base de datos, el desarrollo de la lógica de negocio en el servidor y la construcción de una interfaz de usuario que permita la interacción de forma clara e intuitiva.

En este sentido, el sistema desarrollado permite gestionar de manera centralizada las principales entidades implicadas en la actividad empresarial, tales como clientes, contenedores de aceite, productos, conductores, vehículos, rutas y recogidas. Asimismo, se incluyen las operaciones fundamentales de consulta, inserción, modificación y eliminación de datos (operaciones CRUD), garantizando la integridad y coherencia de la información almacenada mediante un modelo de datos relacional adecuadamente estructurado.

El proyecto también contempla la implementación de un sistema de autenticación de usuarios que regula el acceso a la aplicación, así como un panel de administración desde el cual es posible gestionar los distintos recursos del sistema según los permisos asignados.

De este modo, se establece una base sólida que permite controlar los procesos internos de la empresa y mejorar la organización de las tareas diarias. No obstante, es importante señalar que determinadas funcionalidades avanzadas, como la integración con servicios externos de geolocalización, la visualización de datos en mapas

interactivos, los sistemas de notificaciones automáticas o la implementación de mecanismos avanzados de seguridad, no forman parte del alcance actual del proyecto.

Estas características han sido consideradas como posibles líneas de ampliación futura, con el objetivo de evolucionar el sistema hacia una solución más completa y adaptada a entornos reales de producción.

2. Análisis del sector / mercado

El sector del desarrollo de aplicaciones web se encuentra en constante crecimiento y evolución, impulsado por la transformación digital de empresas y organizaciones en prácticamente todos los ámbitos económicos. En este contexto, la digitalización de procesos empresariales se ha convertido en un elemento clave para mejorar la eficiencia operativa, la trazabilidad de la información y la toma de decisiones basada en datos.

La demanda de soluciones web personalizadas ha aumentado notablemente en los últimos años, especialmente en sectores como la logística, la gestión de residuos, el transporte y los servicios, donde resulta fundamental disponer de herramientas que permitan gestionar grandes volúmenes de información de forma estructurada y accesible. Este tipo de aplicaciones facilita la automatización de tareas, la reducción de errores humanos y la optimización de recursos.

El proyecto desarrollado se enmarca en este contexto, ofreciendo una solución tecnológica que responde a necesidades reales de gestión empresarial mediante el uso de tecnologías web modernas y bases de datos relacionales.

2.1. Prospectiva del Título en el sector

El título de Técnico Superior en Desarrollo de Aplicaciones Web presenta una elevada proyección profesional dentro del mercado laboral actual, debido a la creciente necesidad de perfiles especializados en el desarrollo, mantenimiento y despliegue de aplicaciones web.

Los titulados en DAW pueden desempeñar funciones en distintos ámbitos, tales como el desarrollo frontend y backend, la administración de bases de datos, la integración de servicios web o la implementación de soluciones empresariales a medida. Asimismo, la versatilidad de este perfil permite su adaptación a múltiples sectores, desde empresas tecnológicas hasta organizaciones tradicionales en proceso de digitalización.

2.2. Evolución y tendencia del sector

El sector del desarrollo web ha experimentado una evolución significativa en las últimas décadas, pasando de páginas web estáticas a aplicaciones dinámicas altamente interactivas y orientadas a servicios. Actualmente, las aplicaciones web modernas se caracterizan por el uso de arquitecturas cliente-servidor, APIs REST, frameworks avanzados y tecnologías que permiten una experiencia de usuario fluida y eficiente.

Entre las principales tendencias del sector destacan:

- La adopción de frameworks modernos tanto en frontend como en backend.
- El uso de bases de datos relacionales y no relacionales según las necesidades del proyecto.
- La integración de sistemas de geolocalización y visualización de datos en mapas.
- La automatización de procesos mediante sistemas de alertas y notificaciones.
- La creciente importancia de la seguridad en aplicaciones web y protección de datos.

2.3. Normativa y documentación técnica específica

El desarrollo de aplicaciones web debe cumplir con una serie de normativas y buenas prácticas que garantizan la seguridad, accesibilidad y protección de los datos. En este sentido, uno de los aspectos más relevantes es el cumplimiento de la normativa en materia de protección de datos personales, especialmente el Reglamento General de Protección de Datos (RGPD), que establece las obligaciones en el tratamiento de la información de usuarios y clientes.

Asimismo, se deben tener en cuenta principios de seguridad en el desarrollo de software, como la protección frente a accesos no autorizados, la gestión segura de contraseñas y la validación de datos de entrada. También resulta importante seguir estándares de desarrollo que faciliten el mantenimiento y la escalabilidad del sistema, como el uso de arquitecturas estructuradas y documentación técnica adecuada.

En cuanto a la documentación técnica, el proyecto incluye la definición del modelo de datos, la descripción de las funcionalidades implementadas, así como los aspectos relacionados con la instalación, configuración y uso de la aplicación. Todo ello contribuye a garantizar la comprensión del sistema y su posible evolución futura.

3.2. Proceso de desarrollo software

3.2.1. Fase de análisis

En esta fase se estudian los requisitos funcionales y no funcionales del sistema para, posteriormente, tomar decisiones fundamentadas sobre las herramientas, lenguajes y frameworks que mejor se adaptan a las necesidades del proyecto. En este apartado se detalla y justifica la elección de cada tecnología del stack de desarrollo, buscando siempre el equilibrio entre productividad, rendimiento, mantenibilidad y adecuación al problema a resolver.

La elección del stack tecnológico en este proyecto ha sido una de las decisiones más críticas en la fase de análisis. Un stack bien elegido garantiza la productividad del equipo, la calidad del producto final, la facilidad de mantenimiento a largo plazo y la correcta adecuación a los requisitos del sistema.

Para este proyecto, se ha optado por un stack moderno, robusto y ampliamente respaldado por la industria, basado en tecnologías de código abierto con comunidades activas y abundante documentación. La arquitectura general sigue un modelo cliente-servidor con una clara separación entre el frontend, el backend y la capa de persistencia de datos.

Para el frontend se ha implementado como una Single Page Application (SPA) utilizando el framework Angular. Esto permite una experiencia de usuario fluida y similar a la de una aplicación de escritorio, ya que la página no se recarga completamente durante la navegación. La capa de presentación consume datos de la API REST del backend y se apoya en Bootstrap para garantizar un diseño responsive y una implementación ágil de componentes visuales como modales, formularios y tablas, elementos que aparecen de forma recurrente en la lista de tareas del proyecto.

La capa de Lógica de Negocio (Backend) actúa como intermediaria entre la interfaz de usuario y los datos. Se ha construido con PHP y comunicada con phpmyadmin mediante PDO, conformando una API REST. Se ha optado por PHP por ser lo practicado durante el curso académico y la facilidad de expansión gracias al MVC. La lógica de llamar a todos los controladores con el mismo código y a sus modelos correspondientes, nos permite con un solo index.php expandirnos sin tener que preocuparnos de alterar código anterior, lo que refleja la implementación del patrón MVC en el servidor.

3.2.1.1. Tipos de usuario

En la aplicación se contemplan principalmente tres tipos de usuario:

El usuario **administrador** dispone de acceso completo a todas las funcionalidades de la aplicación. Este tipo de usuario es responsable de la gestión global del sistema, pudiendo crear, modificar y eliminar registros en todas las entidades, tales como clientes, contenedores, productos, conductores, zonas y rutas. Asimismo, tiene acceso al panel de administración, desde donde puede gestionar usuarios y supervisar el funcionamiento general de la aplicación.

Por otro lado, se define el perfil de **usuario estándar**, cuyo acceso está limitado a determinadas funcionalidades operativas. Este tipo de usuario puede consultar información relevante, registrar datos de recogidas y gestionar ciertos elementos del sistema, pero no dispone de permisos para realizar acciones críticas como la eliminación de registros o la modificación de configuraciones globales.

Finalmente, se contempla el perfil de **conductor**, cuyo acceso está restringido a la consulta de información relacionada con su actividad diaria. Este tipo de usuario puede visualizar las rutas asignadas, así como los contenedores asociados a dichas rutas, con el objetivo de facilitar la ejecución de las tareas de recogida. No dispone de permisos para modificar información ni realizar operaciones sobre los datos del sistema.

3.2.1.2. Descripción de requisitos

Requisitos funcionales

Los requisitos funcionales describen las acciones y servicios que el sistema debe proporcionar para satisfacer las necesidades del usuario, ya sea admin, estándar o conductor.

- El sistema debe permitir la autenticación de usuarios mediante inicio de sesión.
-> TODOS USUARIOS
- El sistema debe permitir cerrar sesión de forma segura. -> TODOS USUARIOS
- El sistema debe permitir la creación de nuevos clientes. -> ADMIN Y ESTANDAR
- El sistema debe permitir la consulta de clientes mediante distintos criterios de búsqueda (nombre, CIF/DNI, municipio). -> TODOS
- El sistema debe permitir la modificación de los datos de los clientes existentes.
-> ADMIN
- El sistema debe permitir la consulta de clientes registrados

- El sistema debe permitir registrar y modificar la información de los contenedores (tipo, capacidad, ubicación, estado, periodo de recogida). -> ADMIN
- El sistema debe permitir la gestión de productos (crear, consultar, editar y eliminar). -> ADMIN
- El sistema debe permitir asociar productos a una recogida. -> ADMIN Y ESTANDAR
- El sistema debe permitir registrar recogidas de contenedores. -> ADMIN Y ESTANDAR
- El sistema debe almacenar información detallada de cada recogida (fecha, litros recogidos, estado, incidencias, notas). -> ADMIN Y ESTANDAR
- El sistema debe permitir consultar el historial de recogidas. -> ADMIN Y ESTANDAR
- El sistema debe permitir la creación y gestión de rutas. -> ADMIN Y ESTANDAR
- El sistema debe permitir asignar conductores y vehículos a rutas. -> ADMIN Y ESTANDAR
- El sistema debe permitir asociar contenedores a rutas. -> ADMIN Y ESTANDAR
- El sistema debe permitir consultar las rutas programadas. -> TODOS
- El sistema debe permitir la gestión de conductores (crear, consultar y editar) -> ADMIN Y ESTANDAR
- El sistema debe permitir la gestión de vehículos y sus datos asociados (ITV, mantenimiento, seguros). -> ADMIN Y ESTANDAR
- El sistema debe permitir la gestión de zonas. -> ADMIN Y ESTANDAR
- El sistema debe permitir consultar información filtrada por municipios o zonas. -> TODOS
- El sistema debe disponer de un panel de administración para la gestión global del sistema. -> ADMIN Y ESTANDAR

Requisitos no funcionales

Los requisitos no funcionales definen las características de calidad que debe cumplir el sistema:

- El sistema debe ser accesible a través de navegadores web modernos sin necesidad de instalación adicional.
- El sistema debe ofrecer una interfaz de usuario clara, intuitiva y fácil de usar.
- El sistema debe garantizar tiempos de respuesta adecuados en las operaciones.
- El sistema debe asegurar la protección de datos personales conforme a la normativa vigente.
- El sistema debe implementar mecanismos de autenticación y control de acceso según roles.
- El sistema debe proteger la información frente a accesos no autorizados.

- El sistema debe garantizar la integridad y consistencia de los datos almacenados.
- El sistema debe validar los datos introducidos por el usuario para evitar errores.
- El sistema debe ser escalable, permitiendo la incorporación de nuevas funcionalidades en el futuro.
- El sistema debe estar estructurado de forma modular para facilitar su mantenimiento.
- El sistema debe permitir su despliegue en un entorno servidor estándar.
- El sistema debe estar correctamente documentado para facilitar su uso y mantenimiento.

3.2.1.3. Casos de Uso

A continuación, se detallan los casos de uso solicitados con los datos de entrada y salida añadidos:

CU-01: Iniciar sesión

- **Identificador CU:** CU-01
- **Nombre CU:** Iniciar sesión
- **Actores:** Usuario no autenticado (Admin, Usuario estándar, Conductor)
- **Descripción:** Permite a un usuario acceder al sistema mediante sus credenciales.
- **Prioridad:** 10
- **Precondición:** El usuario dispone de una cuenta activa en la tabla usuarios.
- **Datos de entrada:** Email, contraseña.
- **Datos de salida:** Redirección al panel principal según rol; mensaje de error si falla.
- **Secuencia Normal:**

Usuario	Sistema
1. Accede a la página de inicio de sesión.	
2. Introduce email y contraseña.	
	3. Verifica las credenciales contra la tabla usuarios.
	4. Si son correctas, crea la sesión y redirige al panel correspondiente según el rol.

- **Flujos alternativos:**

Usuario	Sistema
	3.a. Email o contraseña incorrectos: muestra mensaje de error y permite reintentar.

- **Postcondición:** El usuario queda autenticado y accede a las funcionalidades de su rol.
- **CU Relacionados:** CU-02

CU-02: Cerrar sesión

- **Identificador CU:** CU-02
- **Nombre CU:** Cerrar sesión
- **Actores:** Usuario autenticado (cualquier rol)
- **Descripción:** Finaliza la sesión activa del usuario de forma segura.
- **Prioridad:** 9
- **Precondición:** El usuario ha iniciado sesión.
- **Datos de entrada:** Ninguno.
- **Datos de salida:** Redirección a la pantalla de inicio de sesión.
- **Secuencia Normal:**

Usuario	Sistema
1. Pulsa la opción "Cerrar sesión".	
	2. Elimina la sesión y redirige a la página de login.

- **Flujos alternativos:** No aplican.
- **Postcondición:** La sesión queda cerrada; se requiere nuevo inicio de sesión para acceder.
- **CU Relacionados:** CU-01

CU-03: Editar perfil de usuario

- **Identificador CU:** CU-03
- **Nombre CU:** Editar perfil de usuario
- **Actores:** Usuario autenticado (cualquier rol)
- **Descripción:** El usuario modifica sus propios datos personales (nombre, email, teléfono, contraseña).

- **Prioridad:** 7
- **Precondición:** El usuario ha iniciado sesión.
- **Datos de entrada:** Nuevos valores para nombre, email, teléfono, contraseña.
- **Datos de salida:** Confirmación de actualización o mensaje de error.
- **Secuencia Normal:**

Usuario	Sistema
1. Accede a la sección “Mi perfil”.	
	2. Recupera los datos actuales de usuarios.
3. Modifica los campos deseados y guarda.	
	4. Valida los datos (formato email, longitud, etc.).
	5. Actualiza el registro en la tabla usuarios.
	6. Muestra mensaje de confirmación.

- **Flujos alternativos:**

Usuario	Sistema
	4.a. Datos no válidos: muestra errores y pide corrección.
3.b. Cancela la operación: el caso de uso termina sin cambios.	

- **Postcondición:** Los datos del perfil quedan actualizados.
- **CU Relacionados:** –

CU-04: Registrar nuevo cliente

- **Identificador CU:** CU-04
- **Nombre CU:** Registrar nuevo cliente
- **Actores:** Admin, Usuario estándar
- **Descripción:** Da de alta un nuevo cliente en el sistema.
- **Prioridad:** 9
- **Precondición:** El usuario autenticado tiene permisos para crear clientes.

- **Datos de entrada:** PointID, nombre, CIF, teléfono, notas.
- **Datos de salida:** Confirmación de alta o mensaje de error.
- **Secuencia Normal:**

Usuario	Sistema
1. Accede al formulario de nuevo cliente.	
2. Introduce los datos obligatorios y opcionales.	
	3. Valida que el PointID no exista ya en clientes.
	4. Inserta el nuevo registro.
	5. Muestra mensaje de éxito.

- **Flujos alternativos:**

Usuario	Sistema
	3.a. PointID duplicado: muestra error y solicita otro código.
	3.b. Datos obligatorios incompletos: indica campos requeridos.

- **Postcondición:** El cliente queda almacenado en la tabla clientes y puede ser consultado/editado.
- **CU Relacionados:** CU-05, CU-06, CU-07

CU-05: Consultar clientes

- **Identificador CU:** CU-05
- **Nombre CU:** Consultar clientes
- **Actores:** Admin, Usuario estándar, Conductor (solo lectura)
- **Descripción:** Lista y filtra los clientes registrados por nombre, CIF, municipio, etc.
- **Prioridad:** 7
- **Precondición:** Usuario autenticado.
- **Datos de entrada:** Criterios de búsqueda: nombre, CIF, municipio (opcionales).
- **Datos de salida:** Listado de clientes con sus datos básicos.

- **Secuencia Normal:**

Usuario	Sistema
1. Navega a la sección de clientes.	
	2. Recupera y muestra todos los clientes activos (o todos si es admin).
3. Introduce filtros (nombre, CIF, municipio) y pulsa "Buscar".	
	4. Aplica los filtros sobre la tabla clientes y devuelve resultados.

- **Flujos alternativos:**

Usuario	Sistema
	4.a. No se encuentran resultados: muestra mensaje informativo.

- **Postcondición:** Se visualizan los clientes según los criterios.
- **CU Relacionados:** CU-04, CU-06, CU-07

CU-06: Editar cliente

- **Identificador CU:** CU-06
- **Nombre CU:** Editar cliente
- **Actores:** Admin, Usuario estándar
- **Descripción:** Modifica los datos de un cliente existente, respetando la integridad referencial.
- **Prioridad:** 8
- **Precondición:** El cliente existe y el usuario tiene permisos de edición.
- **Datos de entrada:** Nuevos valores para nombre, CIF, teléfono, notas, etc.
- **Datos de salida:** Confirmación de actualización o mensaje de error.
- **Secuencia Normal:**

Usuario	Sistema
1. Selecciona un cliente del listado (CU-05) y pulsa "Editar".	
	2. Carga los datos actuales del cliente.

Usuario	Sistema
3. Modifica los campos necesarios y guarda.	
	4. Valida los nuevos datos (unicidad de PointID si cambia).
	5. Actualiza el registro en clientes.
	6. Muestra mensaje de éxito.

- **Flujos alternativos:**

Usuario	Sistema
	4.a. Datos inválidos: muestra errores y pide corrección.
3.b. Cancela la edición: vuelve al listado sin cambios.	

- **Postcondición:** Los datos del cliente quedan actualizados.
- **CU Relacionados:** CU-05, CU-07

CU-07: Desactivar cliente

- **Identificador CU:** CU-07
- **Nombre CU:** Desactivar cliente
- **Actores:** Admin
- **Descripción:** Desactiva un cliente (borrado lógico), impidiendo futuras operaciones, pero manteniendo el historial. El cliente nunca se elimina físicamente.
- **Prioridad:** 6
- **Precondición:** El cliente existe y está activo. El usuario es administrador.
- **Datos de entrada:** ID del cliente a desactivar.
- **Datos de salida:** Confirmación de desactivación o error.
- **Secuencia Normal:**

Usuario	Sistema
1. Desde la vista de detalle del cliente, pulsa “Desactivar”.	
	2. Solicita confirmación.
3. Confirma la acción.	
	4. Marca el cliente como inactivo (campo activo = FALSE).
	5. Muestra mensaje de éxito.

- **Flujos alternativos:**

Usuario	Sistema
3.a. Cancela: el cliente permanece activo.	

- **Postcondición:** El cliente queda desactivado; no aparece en consultas operativas pero sus contenedores y recogidas asociadas permanecen.

- **CU Relacionados:** CU-05, CU-06

Nota: La tabla clientes actual no incluye campo activo; se asume la existencia de dicha columna para implementar el borrado lógico exigido.

CU-08: Consultar contenedores

- **Identificador CU:** CU-08
- **Nombre CU:** Consultar contenedores
- **Actores:** Admin, Usuario estándar, Conductor (asignados a sus rutas)
- **Descripción:** Lista los contenedores registrados, pudiendo filtrar por cliente, estado, tipo, ubicación, etc.
- **Prioridad:** 7
- **Precondición:** Usuario autenticado.
- **Datos de entrada:** Filtros: ID cliente, tipo de contenedor, activo/inactivo, municipio.
- **Datos de salida:** Listado de contenedores con datos relevantes (tipo, mercancía, dirección, coordenadas, estado).
- **Secuencia Normal:**

Usuario	Sistema
1. Accede a la sección de contenedores.	
	2. Recupera todos los contenedores (según permisos) con sus relaciones (tipo, cliente, dirección, municipio).
3. Aplica filtros opcionales y pulsa "Filtrar".	
	4. Devuelve los contenedores que cumplen los criterios.

- **Flujos alternativos:** Sin resultados → mensaje informativo.
- **Postcondición:** Se visualiza la lista de contenedores.
- **CU Relacionados:** CU-09, CU-10, CU-11

CU-09: Registrar nuevo contenedor

- **Identificador CU:** CU-09
- **Nombre CU:** Registrar nuevo contenedor
- **Actores:** Admin, Usuario estándar
- **Descripción:** Da de alta un nuevo contenedor asociado a un cliente y lo geolocaliza.
- **Prioridad:** 9
- **Precondición:** El cliente asociado existe y está activo. Existen tipos de contenedor en tipo_contenedor.
- **Datos de entrada:** Cliente, tipo de contenedor, tipo legal, período de recogida (días), mercancía, coordenadas (lat/long), dirección completa (dirección, código postal, municipio), fechas inicio/fin.
- **Datos de salida:** Confirmación de creación o mensaje de error.

- **Secuencia Normal:**

Usuario	Sistema
1. Accede al formulario de nuevo contenedor.	
2. Selecciona cliente, tipo de contenedor, rellena campos y coordenadas.	
	3. Valida que el cliente existe y está activo, que el tipo es válido y que los campos obligatorios están completos.
	4. Inserta registro en contenedores con activo = TRUE.
	5. Inserta la dirección asociada en direcciones usando el id_contenedor generado.
	6. Muestra mensaje de éxito.

- **Flujos alternativos:**

Usuario	Sistema
	3.a. Cliente inactivo o tipo no existente: muestra error.
	3.b. Coordenadas fuera de rango: pide corrección.

- **Postcondición:** El contenedor queda registrado y disponible para ser asignado a rutas.
- **CU Relacionados:** CU-08, CU-10, CU-11, CU-16

CU-10: Editar contenedor

- **Identificador CU:** CU-10
- **Nombre CU:** Editar contenedor
- **Actores:** Admin, Usuario estándar
- **Descripción:** Modifica los datos de un contenedor existente (tipo, dirección, coordenadas, estado, etc.).
- **Prioridad:** 8

- **Precondición:** El contenedor existe.
- **Datos de entrada:** Nuevos valores para los campos editables.
- **Datos de salida:** Confirmación de actualización o error.
- **Secuencia Normal:**

Usuario	Sistema
1. Selecciona un contenedor del listado y pulsa "Editar".	
	2. Carga los datos actuales (contenedor + dirección).
3. Modifica campos y guarda.	
	4. Valida los cambios.
	5. Actualiza contenedores y, si procede, direcciones.
	6. Muestra éxito.

- **Flujos alternativos:** Datos inválidos → errores.
- **Postcondición:** Los datos del contenedor quedan actualizados.
- **CU Relacionados:** CU-08, CU-09

CU-11: Desactivar contenedor

- **Identificador CU:** CU-11
- **Nombre CU:** Desactivar contenedor
- **Actores:** Admin
- **Descripción:** Desactiva un contenedor (borrado lógico), impidiendo su inclusión en nuevas rutas.
- **Prioridad:** 6
- **Precondición:** El contenedor está activo.
- **Datos de entrada:** ID del contenedor.
- **Datos de salida:** Confirmación.
- **Secuencia Normal:**

Usuario	Sistema
1. Desde el detalle del contenedor pulsa "Desactivar".	

Usuario	Sistema
	2. Pide confirmación.
3. Confirma.	
	4. Pone activo = FALSE en contenedores.
	5. Muestra mensaje.

- **Flujos alternativos:** Cancelación → sin cambios.
- **Postcondición:** El contenedor queda inactivo; no se asigna a rutas futuras pero su historial permanece.
- **CU Relacionados:** CU-08, CU-10

CU-12: Consultar productos

- **Identificador CU:** CU-12
- **Nombre CU:** Consultar productos
- **Actores:** Admin, Usuario estándar
- **Descripción:** Muestra el catálogo de productos que la empresa entrega a los clientes (aceite, bidones, etc.).
- **Prioridad:** 6
- **Precondición:** Usuario autenticado.
- **Datos de entrada:** Filtro por tipo (opcional).
- **Datos de salida:** Lista de productos con tipo, coste y notas.
- **Secuencia Normal:**

Usuario	Sistema
1. Accede a “Productos”.	
	2. Recupera todos los productos de la tabla productos y los muestra.
3. Opcionalmente filtra por tipo.	
	4. Aplica filtro.

- **Postcondición:** Se visualiza el listado de productos.
- **CU Relacionados:** CU-13, CU-14, CU-15

CU-13: Registrar nuevo producto

- **Identificador CU:** CU-13
- **Nombre CU:** Registrar nuevo producto
- **Actores:** Admin
- **Descripción:** Crea un nuevo tipo de producto en el catálogo.
- **Prioridad:** 7
- **Precondición:** El usuario es administrador.
- **Datos de entrada:** Tipo, coste, notas.
- **Datos de salida:** Confirmación.
- **Secuencia Normal:**

Usuario	Sistema
1. Pulsa “Nuevo producto” en el listado.	
2. Introduce los datos.	
	3. Valida campos obligatorios (tipo).
	4. Inserta en productos.
	5. Muestra éxito.

- **Flujos alternativos:** Datos inválidos → error.
- **Postcondición:** El producto queda disponible para ser asignado a recogidas.
- **CU Relacionados:** CU-12

CU-14: Editar producto

- **Identificador CU:** CU-14
- **Nombre CU:** Editar producto
- **Actores:** Admin
- **Descripción:** Modifica los datos de un producto existente.
- **Prioridad:** 6
- **Precondición:** El producto existe.
- **Datos de entrada:** Nuevo tipo, coste, notas.
- **Datos de salida:** Confirmación.
- **Secuencia Normal:** Similar a edición, actualizando productos.
- **Flujos alternativos:** Validación de datos.
- **Postcondición:** El producto se actualiza.
- **CU Relacionados:** CU-12

CU-15: Eliminar producto

- **Identificador CU:** CU-15
- **Nombre CU:** Eliminar producto
- **Actores:** Admin
- **Descripción:** Elimina un producto del catálogo, siempre que no esté referenciado en ninguna recogida.
- **Prioridad:** 5
- **Precondición:** El producto no está asociado a ningún registro en productos_recogidas.
- **Datos de entrada:** ID del producto.
- **Datos de salida:** Confirmación o mensaje de error si tiene dependencias.
- **Secuencia Normal:**

Usuario	Sistema
1. En el listado de productos, pulsa "Eliminar" sobre uno.	
	2. Verifica que no existan filas en productos_recogidas con ese ID.
	3. Si no hay referencias, elimina el registro.
	4. Muestra éxito.

- **Flujos alternativos:**

Usuario	Sistema
	2.a. Existen recogidas asociadas: muestra error informando que no se puede eliminar.

- **Postcondición:** El producto se elimina o permanece si está en uso.
- **CU Relacionados:** CU-12, CU-17

CU-16: Crear ruta

- **Identificador CU:** CU-16
- **Nombre CU:** Crear ruta
- **Actores:** Admin, Usuario estándar
- **Descripción:** Planifica una nueva ruta de recogida, asignando un conductor, un vehículo, una fecha y uno o varios contenedores a visitar.

- **Prioridad:** 10
- **Precondición:** Existen conductores activos y vehículos disponibles. Los contenedores seleccionados deben estar activos.
- **Datos de entrada:** Conductor, vehículo (matrícula), fecha, notas, lista de IDs de contenedores.
- **Datos de salida:** Confirmación de creación de ruta con el número de contenedores asignados, o errores si falla.
- **Secuencia Normal:**

Usuario	Sistema
1. Accede al formulario de “Nueva ruta”.	
2. Selecciona conductor, vehículo, fecha y notas.	
3. Busca y añade contenedores uno a uno (por ID, cliente o zona).	
	4. Por cada contenedor añadido, verifica que existe y está activo.
5. Una vez seleccionados todos los contenedores deseados, pulsa “Guardar ruta”.	
	6. Inserta la ruta en rutas (id_conductor, matricula, fecha, notas).
	7. Por cada contenedor, inserta una fila en rutas-contenedores con el id_ruta generado.
	8. Muestra mensaje de éxito con el resumen de la ruta.

- **Flujos alternativos:**

Usuario	Sistema
	4.a. Contenedor inactivo o inexistente: muestra error y no lo añade.
	6.a. Faltan campos obligatorios (conductor, vehículo, fecha, al menos un contenedor): muestra error.
5.a. Usuario cancela: no se guarda nada.	

- **Postcondición:** La ruta queda planificada con los contenedores asignados. Se podrán registrar recogidas sobre esos contenedores en esa fecha.
- **CU Relacionados:** CU-17, CU-20, CU-18

CU-17: Registrar recogida (con entrega opcional de productos)

- **Identificador CU:** CU-17
- **Nombre CU:** Registrar recogida de contenedor
- **Actores:** Admin, Usuario estándar
- **Descripción:** Registra los datos de la visita a un contenedor en una ruta concreta, incluyendo la posibilidad de entregar productos al dueño (cliente) en ese momento.
- **Prioridad:** 10
- **Precondición:** La ruta existe y tiene contenedores asignados. El contenedor está activo.
- **Datos de entrada:** ID de ruta, ID de contenedor (de entre los asignados a esa ruta), fecha de recogida (puede heredarse de la ruta), litros recogidos, si fue visitado, si se recogió (booleano), bidones recogidos, bidones entregados, notas; opcionalmente, una lista de productos con sus cantidades.
- **Datos de salida:** Confirmación de registro de recogida y de productos entregados, o errores.
- **Secuencia Normal:**

Usuario	Sistema
1. Selecciona una ruta existente (por fecha, conductor, etc.).	

Usuario	Sistema
	2. Muestra los contenedores asignados a esa ruta.
3. Elige un contenedor para registrar su recogida.	
	4. Muestra formulario con campos de recogida y opción de añadir productos.
5. Introduce litros, marca visitado/recogido, bidones, notas.	
6. Si desea entregar productos, pulsa "Añadir producto".	
7. Selecciona un producto del catálogo e indica cantidad.	
	8. Añade el producto a la lista provisional.
9. Repite 6-7 para más productos.	
10. Pulsa "Guardar recogida".	
	11. Valida que todos los campos sean correctos.
	12. Inserta un registro en recogidas con los datos y el id_ruta correspondiente.
	13. Por cada producto en la lista, inserta una fila en productos_recogidas con el id_recogida generado.
	14. Muestra mensaje de éxito.

- **Flujos alternativos:**

Usuario	Sistema
	4.a. El contenedor no pertenece a la ruta: muestra error.
	11.a. Datos de recogida inválidos (litros negativos, etc.): pide corrección.
	11.b. Producto inexistente o cantidad no válida: error.
10.a. Usuario cancela: no se guarda nada.	

- **Postcondición:** La recogida queda registrada y, si se entregaron productos, la relación queda almacenada. El historial de recogidas del contenedor se actualiza.
- **CU Relacionados:** CU-16, CU-18, CU-12

CU-18: Consultar recogidas

- **Identificador CU:** CU-18
- **Nombre CU:** Consultar recogidas
- **Actores:** Admin, Usuario estándar, Conductor (solo sus rutas)
- **Descripción:** Visualiza el historial de recogidas, pudiendo filtrar por fecha, ruta, contenedor o conductor.
- **Prioridad:** 7
- **Precondición:** Usuario autenticado.
- **Datos de entrada:** Filtros: fecha, id_ruta, id_contenedor, id_conductor (según permisos).
- **Datos de salida:** Listado de recogidas con detalle (litros, visitado, recogida, bidones, notas) y productos entregados.
- **Secuencia Normal:**

Usuario	Sistema
1. Accede a "Recogidas".	
	2. Muestra todas las recogidas permitidas (según rol) ordenadas por fecha.

Usuario	Sistema
3. Aplica filtros.	
	4. Devuelve resultados filtrados uniendo recogidas, rutas, contenedores y productos_recogidas si se solicita.

- **Postcondición:** Se visualizan los registros de recogida.
- **CU Relacionados:** CU-17, CU-16

CU-19: Gestionar conductores (crear, editar, consultar)

- **Identificador CU:** CU-19
- **Nombre CU:** Gestionar conductores
- **Actores:** Admin
- **Descripción:** Permite al administrador dar de alta, modificar y listar los conductores de la empresa.
- **Prioridad:** 8
- **Precondición:** El usuario es administrador.
- **Datos de entrada:** Para crear/editar: nombre, email, teléfono.
- **Datos de salida:** Confirmación o listado.
- **Secuencia Normal:** (se desglosa en subflujos: alta, edición, consulta; similar a clientes/productos)
- **Postcondición:** La tabla conductores se mantiene actualizada.
- **CU Relacionados:** CU-16 (asignación a rutas)

CU-20: Consultar rutas

- **Identificador CU:** CU-20
- **Nombre CU:** Consultar rutas
- **Actores:** Admin, Usuario estándar (todas), Conductor (solo las propias)
- **Descripción:** Lista las rutas planificadas y su estado (contenedores, recogidas realizadas).
- **Prioridad:** 7
- **Precondición:** Usuario autenticado.
- **Datos de entrada:** Filtros: fecha, conductor, vehículo.
- **Datos de salida:** Listado de rutas con resumen (número de contenedores, recogidas completadas).
- **Secuencia Normal:**

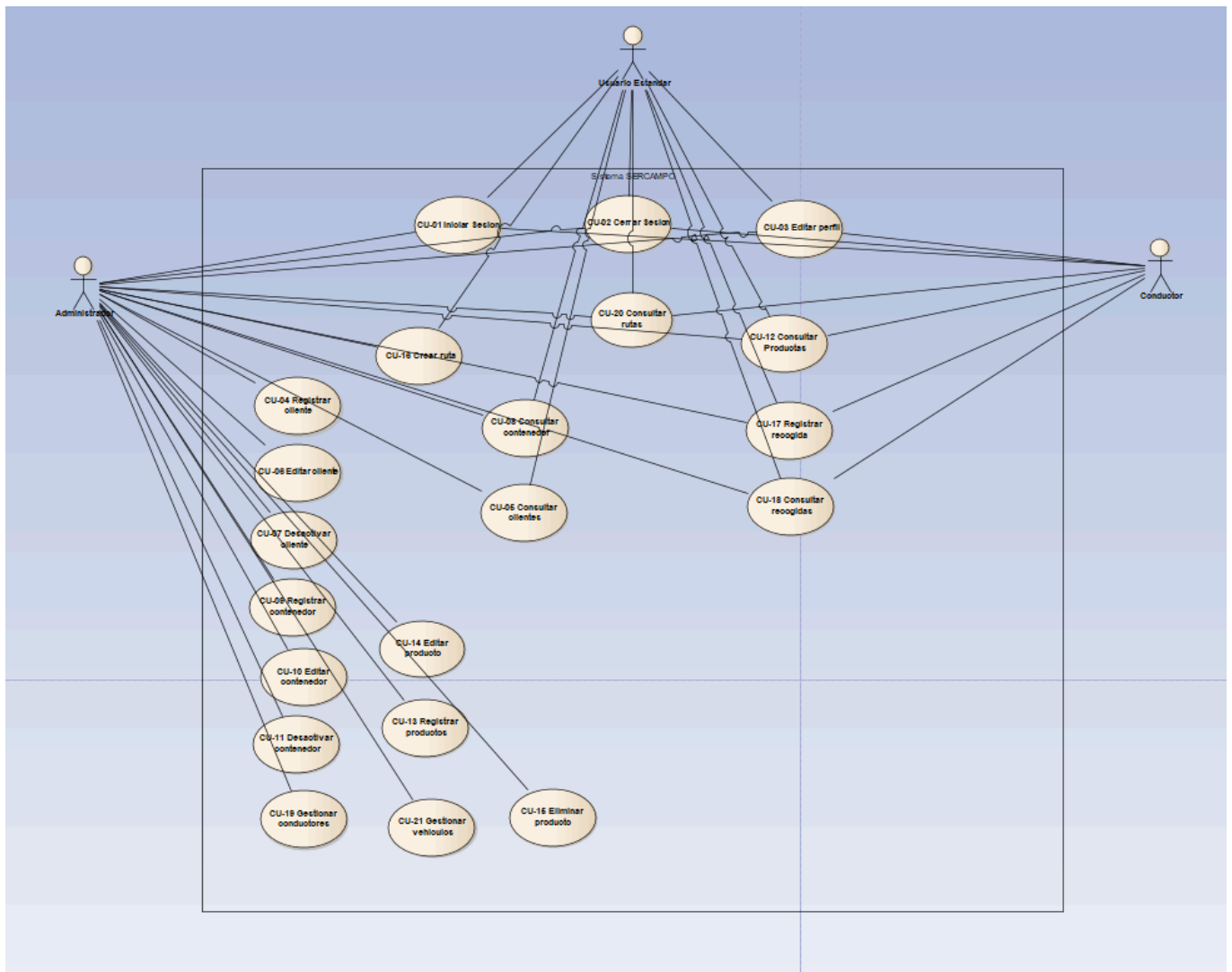
Usuario	Sistema
1. Accede a “Rutas”.	
	2. Consulta rutas y sus relaciones, aplicando filtro de conductor si es rol conductor.
3. Aplica filtros adicionales.	
	4. Muestra resultados.

- **Postcondición:** Se visualizan las rutas.
- **CU Relacionados:** CU-16, CU-17

CU-21: Gestionar vehículos (alta, edición, consulta)

- **Identificador CU:** CU-21
- **Nombre CU:** Gestionar vehículos
- **Actores:** Admin
- **Descripción:** Administra el parque de vehículos (matrícula, modelo, ITV, mantenimiento, seguro).
- **Prioridad:** 7
- **Precondición:** Existen pólizas de seguro en seguros_vehiculos para asociar.
- **Datos de entrada:** Crear/editar: matrícula, modelo, fechas, taller, matrícula, num_poliza, etc.
- **Datos de salida:** Confirmación de operaciones.
- **Postcondición:** Los vehículos están disponibles para ser asignados a rutas.
- **CU Relacionados:** CU-16

La representación de los mismos en el programa Enterprise Architect 9, es el siguiente:





Tipografía

Montserrat
Montserrat
Montserrat
Montserrat
Montserrat
Montserrat

La tipografía Montserrat es una fuente sans serif geométrica diseñada por Julieta Ulanovsky, inspirada en la señalética urbana del barrio de Montserrat en Buenos Aires.

Forma parte del catálogo de Google Fonts, lo que la convierte en una opción accesible, versátil y ampliamente utilizada en entornos digitales.

Claridad y legibilidad Montserrat presenta formas limpias, abiertas y proporciones equilibradas, lo que facilita la lectura tanto en pantallas como en formatos impresos. Funciona especialmente bien en títulos, subtítulos y bloques de texto corto.	Estética contemporánea Su construcción geométrica transmite modernidad, orden y profesionalidad. Es una elección coherente para marcas que buscan una identidad actual, tecnológica o minimalista.	Neutralidad con personalidad Aunque es una fuente neutra y funcional, mantiene ciertos rasgos distintivos (como sus formas redondeadas y proporciones geométricas) que aportan carácter sin comprometer la legibilidad.
Versatilidad tipográfica Cuenta con una amplia gama de pesos (de Thin a Black), lo que permite crear jerarquías visuales claras dentro del diseño sin necesidad de combinar muchas fuentes distintas.	Accesibilidad y rendimiento Al estar disponible en Google Fonts, es gratuita, fácil de integrar en web y optimizada para carga rápida, lo que mejora la experiencia de usuario y el rendimiento del sitio.	

MONTSERRAT

Header 2, 36px Regular

Header 3, 30px Regular

Text, 24px, Regular

Colores Principales

La identidad cromática del proyecto se basa en una paleta equilibrada que combina un color principal con tonos funcionales y neutros, favoreciendo tanto la estética como la usabilidad.

En conjunto, la paleta combina armonía cromática (equilibrio entre tonos fríos, cálidos y neutros) con criterios funcionales, permitiendo construir una interfaz visualmente consistente, accesible y orientada a la interacción.

Principal
F1F1F1

FF9050

4D5B9F

IDENTIDAD CROMÁTICA

El azul (#4D5B9F) actúa como color principal de marca. Transmite confianza, estabilidad y profesionalidad, por lo que se utiliza como eje visual del diseño y en elementos clave de identidad.



El naranja (#FF9050) se emplea como color de acento. Su carácter cálido y energético lo hace ideal para destacar elementos interactivos como estados hover y cabeceras de tablas, guiando la atención del usuario de forma intuitiva.



El blanco (#F1F1F1) funciona como base del sistema. Aporta limpieza, claridad y amplitud visual, facilitando la lectura y permitiendo que el resto de colores destaquen sin generar saturación.



El verde (#198754) y el rojo (#DC3545) cumplen una función semántica dentro de la interfaz. El verde se asocia a acciones positivas o de confirmación, mientras que el rojo indica acciones críticas, advertencias o cancelaciones.



LOGOTIPO



Desde el punto de vista visual, el uso del azul, verde y amarillo responde a una estrategia simbólica alineada con el sector y la identidad de la empresa.

El azul aporta estabilidad, confianza y profesionalidad, funcionando como elemento estructural dentro del logotipo y reforzando la percepción de marca sólida.

El verde introduce una asociación directa con la naturaleza, el crecimiento y la sostenibilidad, conectando con el entorno agrícola y el concepto de campo.

Por su parte, el amarillo actúa como elemento de acento, evocando la luz solar y transmitiendo energía, vitalidad y cercanía.

Iconos

El sistema de iconografía del proyecto se basa en el uso de Bootstrap Icons, una biblioteca de iconos vectoriales diseñada para integrarse de forma nativa con el framework Bootstrap 5.

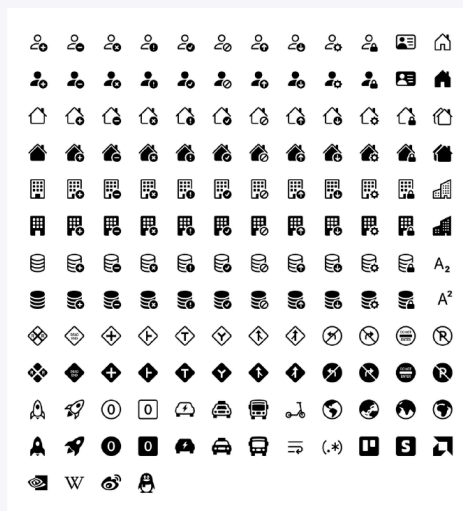
Desde el punto de vista formal, los iconos presentan un estilo lineal y geométrico, con trazos consistentes y proporciones equilibradas. Esta uniformidad favorece la integración con la tipografía Montserrat y el resto del sistema visual, manteniendo una estética moderna y limpia alineada con la interfaz.

A nivel funcional, el uso de Bootstrap Icons permite reforzar la comunicación visual mediante elementos reconocibles que reducen la carga cognitiva del usuario. Los iconos actúan como apoyo semántico en acciones, estados y navegación, facilitando la identificación rápida de funcionalidades sin depender exclusivamente del texto.

En términos técnicos, al tratarse de iconos en formato vectorial (SVG), garantizan escalabilidad sin pérdida de calidad y una correcta adaptación a diferentes resoluciones y dispositivos. Además, su implementación es ligera y optimizada, contribuyendo al rendimiento general de la aplicación.

Desde la perspectiva de accesibilidad, los iconos se utilizan como complemento y no como único medio de comunicación, asegurando que la información relevante también esté disponible mediante texto o atributos accesibles (como etiquetas ARIA). Esto permite cumplir con buenas prácticas definidas en estándares como las WCAG.

Asimismo, la integración con Bootstrap facilita la consistencia en estados interactivos (hover, activo, deshabilitado), permitiendo aplicar estilos de forma uniforme y predecible en toda la interfaz. En resumen, la utilización de Bootstrap Icons contribuye a construir una interfaz más intuitiva, consistente y accesible, alineada tanto con los principios de diseño del sistema como con las necesidades funcionales del proyecto.



3.2.2. Fase de desarrollo

3.2.2.1. Base de datos

La base de datos desarrollada en este proyecto tiene como objetivo la gestión integral de un sistema de recogida y logística de contenedores, permitiendo almacenar y organizar información relacionada con clientes, contenedores, rutas, vehículos, conductores, productos y recogidas realizadas.

El diseño se ha realizado siguiendo un modelo relacional, garantizando la coherencia de los datos, la integridad referencial y la optimización de las consultas.

Para la implementación de la base de datos se ha utilizado MySQL, uno de los sistemas de gestión de bases de datos relacionales más extendidos en el desarrollo de aplicaciones web y de escritorio.

Además, se ha trabajado con herramientas de administración como phpMyAdmin, incluida en XAMPP, que permite gestionar la base de datos de manera visual, ejecutar consultas SQL y verificar la estructura de las tablas de forma rápida e intuitiva.

En conjunto, estas tecnologías han permitido desarrollar una base de datos eficiente, escalable y preparada para su utilización.

3.2.2.1.1. Análisis de requisitos de datos de la aplicación

En este apartado se describe el diseño de la base de datos. Se identifican las entidades, sus atributos, así como las relaciones existentes entre ellas y sus correspondientes cardinalidades. Finalmente, se presenta el modelo relacional resultante.

a. Identificación de entidades e interrelaciones entre entidades

A partir del análisis de los requisitos de la aplicación, se han identificado las siguientes entidades principales:

- Usuarios
- Clientes
- Contenedores
- Tipo_contenedor
- Productos

- Conductores
- Vehículos
- Rutas
- Recogidas
- Municipios
- Zonas
- Direcciones
- Seguros_vehículos

Relaciones entre entidades

- Un cliente puede tener varios contenedores (relación 1:N).
- Cada contenedor pertenece a un único cliente y es de un tipo de contenedor.
- Un contenedor tiene asociada una única dirección (relación 1:1).
- Un municipio pertenece a una zona, pero una zona puede tener muchos municipios(relación 1:N).
- Una ruta es realizada por un conductor y puede usar un vehículo.
- Una ruta incluye varios contenedores y un contenedor puede estar en varias rutas en diferentes fechas (relación N:M).
- Una recogida se realiza sobre un contenedor dentro de una ruta.
- Una recogida puede incluir varios productos (relación N:M).
- Un vehículo debe tener asociado un seguro.

b. identificación de atributos de cada entidad

A continuación, se describen los principales atributos de cada entidad:

Usuarios

- **id_usuario:** Identificador único del usuario.
- **nombre:** Nombre del usuario.
- **email:** Correo electrónico utilizado para acceso y contacto.
- **telefono:** Teléfono de contacto.
- **password:** Contraseña cifrada del usuario.
- **rol:** Rol dentro del sistema (administrador, usuario o conductor).

Clientes

- **id_cliente**: Identificador único del cliente.
- **PointID**: Código identificativo del cliente, es un código inventando por la empresa.
- **nombre**: Nombre o razón social.
- **cif**: Identificación fiscal del cliente.
- **telefono**: Teléfono de contacto.
- **notas**: Información adicional.

Contenedores

- **id_contenedor**: Identificador único del contenedor.
- **id_tipo_contenedor**: Identificador del tipo de contenedor.
- **id_cliente**: Identificador único del cliente propietario.
- **tipo_legal**: Define si es de tipo HORECA, EESS, CONTENEDOR DE VÍA PÚBLICA.
- **periodo_recogida_dias**: Frecuencia de recogida esperada.
- **latitud / longitud**: Ubicación geográfica exacta.
- **inicio**: Fecha de inicio del contrato con la empresa.
- **fin**: Fecha de finalización del contrato, si existe, activo deberá ser false.
- **activo**: Indica si el contenedor está operativo y hay que recogerlo o no.

Tipo_contenedor

- **id_tipo_contenedor**: Identificador del tipo.
- **tipo**: Nombre del tipo de contenedor.
- **capacidad**: Capacidad en litros del contenedor.
- **notas**: Información adicional.

Productos

- **id_producto**: Identificador del producto.
- **tipo**: Tipo de producto entregado al cliente.
- **coste**: Coste asociado.
- **notas**: Información adicional.

Conductores

- **id_conductor**: Identificador del conductor.
- **nombre**: Nombre del conductor.
- **email**: Correo electrónico.
- **telefono**: Teléfono de contacto.

Vehículos

- **matricula**: Identificador único del vehículo.

- **modelo:** Modelo del vehículo.
- **fecha_itv:** Fecha de inspección técnica.
- **fecha_mantenimiento:** Último mantenimiento en el taller.
- **precio_mantenimiento:** Coste de dicho mantenimiento.
- **taller_mantenimiento:** Taller que realizó ese mantenimiento.
- **num_poliza:** Seguro único asociado.
- **num_bastidor:** Número de bastidor.

Rutas

- **id_ruta:** Identificador de la ruta.
- **id_conductor:** Conductor asignado a esa ruta.
- **matricula:** Vehículo utilizado en esa ruta.
- **fecha:** Fecha de la ruta.
- **notas:** Observaciones.

Recogidas

- **id_recogida:** Identificador de la recogida.
- **id_contenedor:** Contenedor asociado.
- **fecha:** Fecha de la recogida.
- **id_ruta:** Ruta asociada.
- **litros_recogidos:** Cantidad recogida, el contenedor puede no estar lleno.
- **visitado:** Indica si se visitó el contenedor, puede estar en la ruta pero no se visitó.
- **recogida:** Indica si se realizó la recogida, puede visitarse y no recogerse, por la razón que sea.
- **bidones_recogidos:** Número de bidones retirados, puede haber más de un bidón en un bar, o varios contenedores en una misma dirección, y cuentan como el mismo.
- **bidones_entregados:** Número de bidones entregados, puede dejarse una cantidad distinta a la que había.
- **notas:** Información adicional.

Seguros_Vehiculos

- **num_poliza:** Identificador único del seguro, un seguro es solo de un vehículo.
- **tel_emergencias:** Teléfono de contacto del seguro en caso de emergencia, asistencia en carretera.
- **tel_contacto:** Teléfono de contacto del seguro en caso de tener que dar un parte o contactarlos.
- **empresa:** empresa del seguro.

Municipios:

- **id_municipio:** Identificador único de cada municipio.
- **municipio:** nombre de municipio.
- **provincia:** provincia donde se encuentra el municipio
- **pais:** país del municipio, por defecto será España
- **id_zona:** identificador de la zona asociada.

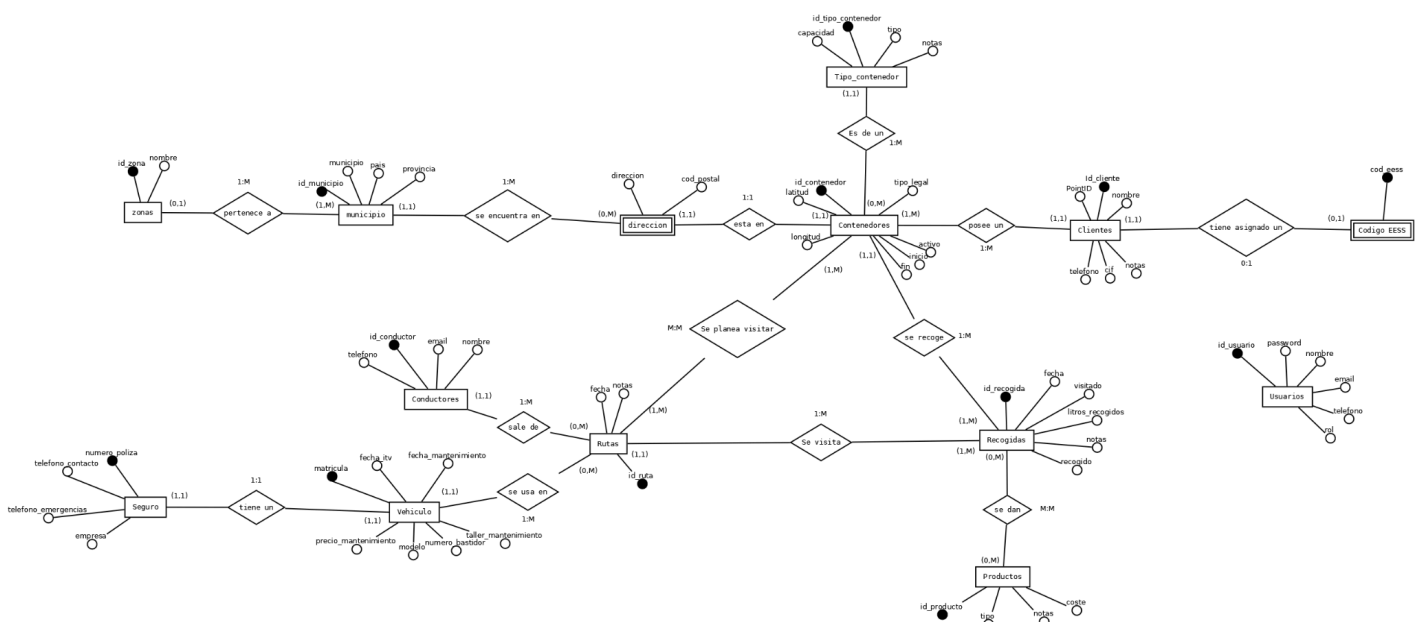
Direcciones:

- **id_contenedor:** Identificador del contenedor que se encuentra en esa dirección.
- **direccion:** nombre o datos de la calle o situación del contenedor. No es una dirección como tal, pueden ser indicaciones que entienden los constructores. Es una cadena de texto.
- **cod_postal:** código postal.
- **id_municipio:** Identificador único del municipio asociado a la dirección.

Codigos_EESS

- **cod_eess:** Identificador único de los clientes que son estaciones de servicio.
- **id_cliente:** id del cliente asociado a este tipo de contenedor.

3.2.2.1.2 Diseño lógico de datos



3.2.2.1.3 Paso del modelo lógico (E/R) al modelo relacional (tablas)

A partir del modelo E/R se han definido las tablas en la base de datos, asignando a cada atributo un tipo de dato adecuado.

a. Identificación de atributos de cada tabla, sus tipos y tamaños

Tabla: usuarios

- id_usuario: INT, clave primaria, autoincremental.
- nombre: VARCHAR(255), almacena el nombre del usuario.
- email: VARCHAR(255), valor único para identificación del usuario.
- telefono: VARCHAR(15), número de teléfono.
- password: VARCHAR(255), contraseña cifrada.
- rol: ENUM ('admin', 'user', 'conductor'), define el tipo de usuario.

Tabla: clientes

- id_cliente: INT, clave primaria, autoincremental.
- PointID: VARCHAR(20), valor único identificativo.
- nombre: VARCHAR(255), nombre o razón social.
- cif: VARCHAR(30), identificación fiscal.
- telefono: VARCHAR(15), número de contacto.
- notas: VARCHAR(255), información adicional.

Tabla: tipo_contenedor

- id_tipo_contenedor: INT, clave primaria, autoincremental.
- tipo: VARCHAR(50), nombre del tipo de contenedor.
- capacidad: INT, capacidad del contenedor.
- notas: VARCHAR(255), observaciones adicionales.

Tabla: contenedores

- id_contenedor: INT, clave primaria, autoincremental.
- id_tipo_contenedor: INT, clave foránea.
- id_cliente: INT, clave foránea.
- tipo_legal: ENUM ('Horeca', 'EESS Repsol', 'Contenedor').
- periodo_recogida_dias: INT, frecuencia de recogida.
- mercancia: VARCHAR(255), tipo de contenido.
- latitud: DECIMAL(10,8), coordenada geográfica.
- longitud: DECIMAL(11,8), coordenada geográfica.
- inicio: DATE, fecha de inicio.
- fin: DATE, fecha de finalización.
- activo: BOOLEAN, estado del contenedor.

Tabla: productos

- id_producto: INT, clave primaria, autoincremental.
- tipo: VARCHAR(50), tipo de producto.
- coste: DECIMAL(7,2), coste asociado.
- notas: VARCHAR(255), información adicional.

Tabla: conductores

- id_conductor: INT, clave primaria, autoincremental.
- nombre: VARCHAR(100), nombre del conductor.
- email: VARCHAR(255), correo electrónico.
- telefono: VARCHAR(20), teléfono de contacto.

Tabla: zonas

- id_zona: INT, clave primaria.
- nombre: VARCHAR(255), nombre de la zona.

Tabla: municipios

- id_municipio: INT, clave primaria, autoincremental.
- municipio: VARCHAR(50), nombre del municipio.
- provincia: VARCHAR(50), provincia.
- pais: VARCHAR(50), país.
- id_zona: INT, clave foránea.

Tabla: direcciones

- id_contenedor: INT, clave primaria y foránea.
- direccion: VARCHAR(255), dirección completa.
- cod_postal: VARCHAR(10), código postal.
- id_municipio: INT, clave foránea.

Tabla: seguros_vehiculos

- num_poliza: VARCHAR(50), clave primaria.
- tel_emergencias: VARCHAR(20), teléfono de emergencias.
- tel_contacto: VARCHAR(20), teléfono de contacto.
- empresa: VARCHAR(255), compañía aseguradora.

Tabla: vehiculos

- matricula: VARCHAR(10), clave primaria.
- modelo: VARCHAR(255), modelo del vehículo.
- fecha_itv: DATE, fecha de inspección técnica.
- fecha_mantenimiento: DATE, fecha del último mantenimiento.
- precio_mantenimiento: DECIMAL(10,2), coste del mantenimiento.
- taller_mantenimiento: VARCHAR(255), taller asociado.
- num_poliza: VARCHAR(50), clave foránea.
- num_bastidor: VARCHAR(20), número de bastidor.

Tabla: rutas

- id_ruta: INT, clave primaria, autoincremental.
- id_conductor: INT, clave foránea.
- matricula: VARCHAR(10), clave foránea.
- fecha: DATE, fecha de la ruta.
- notas: VARCHAR(255), observaciones.

Tabla: rutas_contenedores

- id_ruta: INT, clave primaria y foránea.
- id_contenedor: INT, clave primaria y foránea.

Tabla: recogidas

- id_recogida: INT, clave primaria, autoincremental.
- id_contenedor: INT, clave foránea.
- fecha: DATE, fecha de la recogida.
- id_ruta: INT, clave foránea.
- litros_recogidos: INT, cantidad recogida.
- visitado: BOOLEAN, indica si se visitó el contenedor.
- recogida: BOOLEAN, indica si se realizó la recogida.
- bidones_recogidos: INT, número de bidones retirados.
- bidones_entregados: INT, número de bidones entregados.
- notas: VARCHAR(255), observaciones adicionales.

Tabla: productos_recogidas

- id_recogida: INT, clave primaria y foránea.
- id_producto: INT, clave primaria y foránea.
- cantidad: INT, cantidad del producto recogido.

Tabla: codigos_eess

- cod_eess: INT, clave primaria.
- id_cliente: INT, clave foránea.

*b. Identificación de claves principales y claves candidatas***Claves primarias (PK):**

- usuarios: id_usuario
- clientes: id_cliente
- contenedores: id_contenedor
- rutas: id_ruta
- recogidas: id_recogida
- productos: id_producto
- vehículos: matricula
- seguros_vehiculo: num_poliza
- conductores: id_conductor
- zona: id_zona
- municipios: id_municipio
- tipo_contenedor: id_tipo_contenedor
- código_eess: cod_eess

Claves compuestas:

- rutas_contenedores: (id_ruta, id_contenedor)
- productos_recogidas: (id_recogida, id_producto)

Claves foráneas (FK):

- contenedores.id_cliente que viene de clientes
- contenedores.id_tipo_contenedor que viene de tipo_contenedor
- rutas.id_conductor que viene de conductores
- recogidas.id_ruta que viene de rutas
- municipios.id_zona que viene de zonas
- vehículos.num_poliza que viene de seguros_vehiculos

Claves candidatas:

- usuarios.email (único)
- clientes.PointID (único)

c. Identificación de las reglas de integridad y seguridad de la base de datos

En el diseño de la base de datos se han definido una serie de reglas de integridad y mecanismos de seguridad con el objetivo de garantizar la consistencia, validez y protección de la información almacenada.

Todas las tablas disponen de una clave primaria (PRIMARY KEY), como:

- id_usuario en usuarios
- id_cliente en clientes
- id_contenedor en contenedores
- id_ruta en rutas
- id_recogida en recogidas

Estas claves:

- Son únicas
- No pueden tener valores nulos (NOT NULL)
- En muchos casos son autoincrementales, garantizando su generación automática

En tablas intermedias como:

- rutas_contenedores
- productos_recogidas

se utilizan claves primarias compuestas, evitando duplicidades en relaciones N:M.

Se han definido múltiples FOREIGN KEY, por ejemplo:

- contenedores.id_cliente en referencia a clientes.id_cliente
- contenedores.id_tipo_contenedor en referencia a tipo_contenedor.id_tipo_contenedor
- rutas.id_conductor en referencia a conductores.id_conductor
- recogidas.id_ruta en referencia a rutas.id_ruta

Se han aplicado distintas restricciones según las necesidades:

- **CASCADE:**
 - En relaciones donde los datos dependen totalmente del principal
 - Ejemplo: si se elimina un cliente, se eliminan sus contenedores
- **RESTRICT:**
 - Impide eliminar registros si existen dependencias

- Ejemplo: no se puede eliminar un tipo de contenedor si está en uso
- **SET NULL:**
 - Se mantiene el registro, pero se elimina la referencia
 - Ejemplo: en vehículos, si se elimina un seguro, el campo queda a NULL

Estas decisiones permiten mantener la coherencia de los datos sin perder información relevante.

Además de las restricciones técnicas, se contemplan reglas relacionadas con la lógica de la aplicación:

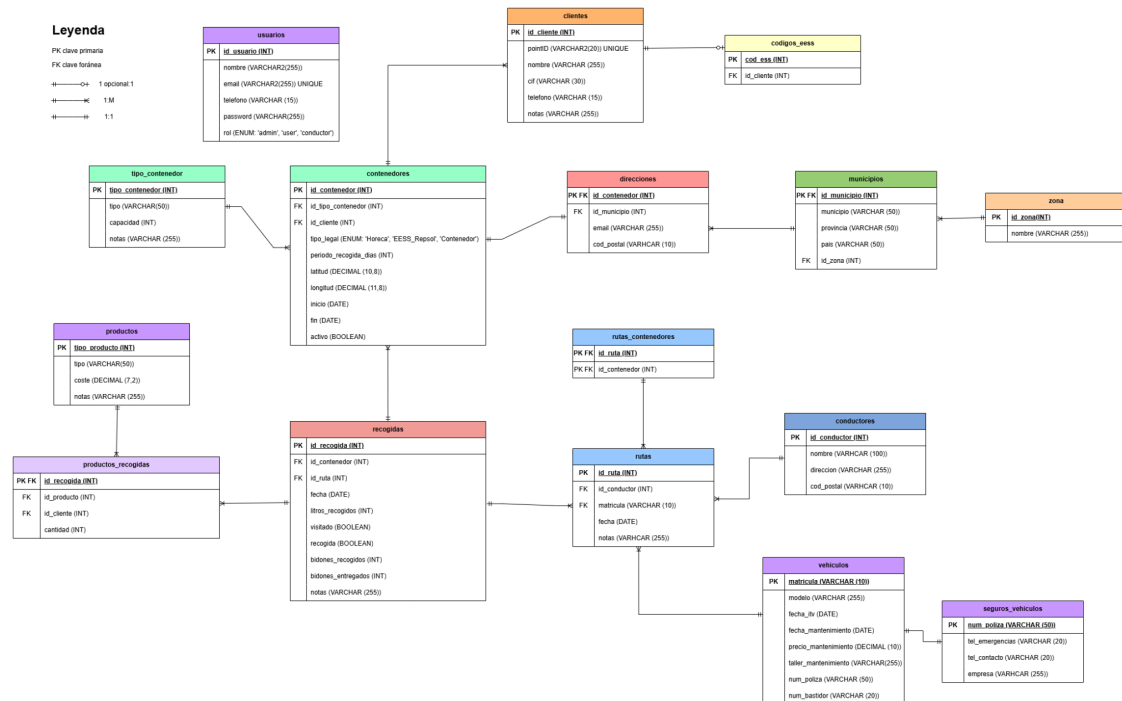
- Un contenedor debe estar asociado a un cliente válido
- Una recogida no puede existir sin un contenedor asociado
- Las rutas deben tener un conductor asignado
- La relación entre rutas y contenedores evita duplicados mediante clave compuesta
- Los productos se registran con cantidades asociadas a cada entrega

Estas reglas garantizan que los datos reflejen correctamente la realidad del sistema.

d. Modelo relacional (tablas)

-- Usuarios: id_usuario(PK), nombre, email, telefono, password, rol, foto
 -- Clientes: id_cliente(PK), POINTID, nombre, cif, telefono, notas
 -- Tipo_contenedor: id_tipo_contenedor(PK), tipo, capacidad, notas
 -- Contenedores: id_contenedor(PK), id_tipo_contenedor(FK), id_cliente(FK), tipo_legal, periodo_recogida_dias, mercancia, latitud, longitud, inicio, fin, activo
 -- Productos: id_productos(PK), tipo, coste, notas.
 -- Conductores: id_conductor(PK), nombre, email, telefono.
 -- Zonas: id_zona(PK), nombre.
 -- Municipios: id_municipio(PK), municipio, provincia, pais, id_zona(FK).
 -- Direcciones: id_contenedor(PK FK), dirección, cod_postal, id_municipio(FK).
 -- Seguros_vehiculo: num_poliza(PK), tel_emergencias, tel_contacto, empresa.
 -- Vehiculos: matricula(PK), modelo, fecha_itv, fecha_mantenimiento, precio_mantenimiento, taller_mantenimiento, num_poliza(FK), num_bastidor.
 -- Rutas: id_ruta(PK), id_conductor(FK), matricula(FK), notas, fecha.
 -- Rutas-contenedores: id_ruta(PK FK), id_contenedor(PK FK).
 -- Recogidas: id_recogida(PK), id_contenedor(FK), fecha, id_ruta(FK), litros_recogidos, visitado, recogida, notas.

- Productos_recogidas: id_recogida(PK FK), id_producto(PK FK), cantidad
- Codigos_eess: cod_eess(PK), id_cliente(FK)



3.2.2.1.4 Aplicación de reglas de normalización al modelo relacional

La normalización es un proceso que busca optimizar el diseño de base de datos eliminando redundancias y garantizando la consistencia de los datos.

Primera Forma Normal (1FN)

La Primera Forma Normal establece que:

- Todos los atributos deben contener valores atómicos (no repetitivos ni multivaluados).
- Cada campo debe contener un único valor por registro.
- Cada registro debe ser único.

En la base de datos diseñada:

- Todas las tablas cumplen la 1FN, ya que no existen listas ni conjuntos de valores en un mismo campo.

- Por ejemplo, en la tabla usuarios, campos como email o teléfono contienen un único valor por registro. En la tabla recogidas, los campos como litros_recogidos o fecha almacenan valores individuales, evitando estructuras repetidas.

Segunda Forma Normal (2FN)

La Segunda Forma Normal establece que:

- Debe cumplirse la 1FN.
- Todos los atributos no clave deben depender completamente de la clave primaria (sin dependencias parciales).

En el modelo:

- Las tablas con clave primaria simple (por ejemplo, clientes, usuarios, vehiculos) cumplen automáticamente la 2FN.
- En las tablas con claves compuestas, como:
 - rutas_contenedores
 - productos_recogidas

se ha garantizado que no existan atributos que dependan solo de una parte de la clave.

- Por ejemplo: En productos_recogidas, el atributo cantidad depende de la combinación de id_recogida e id_producto, no de uno solo de ellos.

Tercera Forma Normal (3FN)

La Tercera Forma Normal establece que:

- Debe cumplirse la 2FN.
- No deben existir dependencias transitivas entre atributos no clave.

En el diseño:

- Se han separado entidades para evitar redundancia de datos.
- Por ejemplo:
 - La información de zonas se almacena en una tabla independiente y no dentro de municipios.
 - Los tipos de contenedor están en una tabla separada para evitar repetir datos en cada contenedor.
 - Los seguros de vehículos se gestionan en una tabla independiente, evitando duplicar información en vehículos.

Esto permite que los datos dependan únicamente de la clave primaria de cada tabla.

Gracias a la aplicación de las formas normales:

- Se ha reducido la redundancia de datos en el sistema.
- Se evita la duplicación innecesaria de información.
- Se mejora la integridad y consistencia de los datos.

- Se facilita el mantenimiento y escalabilidad de la base de datos.

En conclusión, el modelo relacional diseñado cumple hasta la Tercera Forma Normal (3FN), lo que garantiza un diseño óptimo y eficiente para el sistema.

3.2.2.1.5 Tipos de datos para el sistema gestor seleccionado

En el desarrollo de la base de datos se ha utilizado el sistema gestor MySQL, el cual proporciona una amplia variedad de tipos de datos que permiten representar de forma eficiente la información del sistema.

La elección de los tipos de datos se ha realizado en función de la naturaleza de cada atributo, buscando optimizar el almacenamiento, garantizar la integridad de los datos y facilitar su manipulación.

Tipos de datos numéricos

Los tipos numéricos se utilizan para almacenar valores enteros o decimales.

- **INT:** Se utiliza para identificadores y valores numéricos sin decimales, como:
 - id_usuario
 - id_cliente
 - id_contenedor
 - id_ruta

Este tipo permite almacenar números enteros con un rendimiento eficiente en búsquedas y relaciones.
- **DECIMAL:** Se utiliza para valores numéricos con decimales y precisión fija, como:
 - coste de productos
 - precio_mantenimiento
 - coordenadas geográficas (latitud y longitud)

Este tipo garantiza precisión en valores monetarios y medidas.
- **BOOLEAN/TINYINT:** Se utiliza para valores lógicos (verdadero/falso), como:
 - activo en contenedores
 - visitado y recogida en recogidas

Tipos de datos de texto

Los tipos de texto permiten almacenar cadenas de caracteres con distinta longitud.

- **VARCHAR(n):** Es el tipo más utilizado en el sistema, ya que permite almacenar textos de longitud variable. Se utiliza en:
 - nombres (usuarios, clientes, conductores)
 - emails

- teléfonos
- direcciones
- notas

El tamaño se ha ajustado en cada caso según la necesidad (por ejemplo, VARCHAR(255) para textos generales y VARCHAR(15–50) para campos más específicos).

- **ENUM:** Se utiliza para valores predefinidos y controlados, evitando errores de entrada. Por ejemplo:
 - rol en usuarios: ('admin', 'user', 'conductor')
 - tipo_legal en contenedores: ('Horeca', 'EESS Repsol', 'Contenedor')
 Este tipo asegura integridad de dominio.

Tipos de datos de fecha

- **DATE:** Se utiliza para almacenar fechas sin información horaria, como:
 - fecha de rutas
 - fecha de recogidas
 - fecha_itv de vehículos
 - inicio y fin de contenedores

Este tipo permite realizar comparaciones y cálculos temporales de forma eficiente.

En conclusión, la selección de tipos de datos en el sistema ha sido realizada siguiendo criterios de eficiencia, integridad y adecuación a la información almacenada. MySQL proporciona la flexibilidad necesaria para representar tanto datos estructurados como relaciones complejas entre entidades, lo que permite un diseño robusto y escalable.

3.2.2.1.6 Scripts de creación de tablas e inserciones iniciales

El proceso comienza con la creación de la base de datos, asegurando su existencia previa y configurando la codificación de caracteres para soportar textos en diferentes idiomas.

Se define la base de datos con el nombre “sercampo_db”, utilizando la codificación utf8mb4, lo que permite almacenar correctamente caracteres especiales y garantizar compatibilidad internacional.

El script está incluido en los anexos de la memoria, en los archivos adjuntos así como en el enlace a github de la aplicación.

3.2.2.2. Servidor

La parte de servidor, se ha desarrollado en php y nos comunicamos con ella a través de peticiones a una API.

3.2.2.2.1. Lista de funciones en php

Controladores

adminController

- traerProductos(\$dato_producto)
- actualizarProducto(\$dato_producto)
- actualizarConductor(\$datos_conductor)
- nuevoConductor(\$datos_conductor)
- buscadorVehiculo(\$datos_vehiculo)
- actualizarVehiculo(\$datos_vehiculo)
- nuevoVehiculo(\$datos_vehiculo)
- buscadorUsuario(\$datos_usuario)
- actualizarUsuario(\$datos_usuario)
- nuevoUsuario(\$datos_usuario)
- traerContenedores(\$dato_contenedor)
- actualizarContenedor(\$dato_contenedor)

clientesController

- obtenerClientes()
- obtenerClienteGeneral(\$datos)
- obtenerClienteEspecifico(\$datos)
- nuevoCliente(\$datos)
- actualizarCliente(\$datos)
- actualizarEstadoCliente(\$datos)

loginController

- procesarLogin(\$datos_login)
- traerDatos(\$datos_login)

recogidasController

- traerHorecasPorNombre(\$datoBuscador)
- nuevaRecogidaHoreca(\$datosRecogida)
- traerContenedorPorDato(\$datoBuscador)
- nuevaRecogidaContenedor(\$datosRecogida)
- traerEESSPorDato(\$datoBuscador)
- nuevaRecogidaEESS(\$datosRecogida)
- traerIdRuta(\$datosRecogida)
- traerRecogidasFiltradas(\$filtros)
- cargarConductores()
- cargarProvincias()

rutasController

- buscadorParaRutas(\$datoBuscador)
- buscadorConductor(\$datoConductor)
- buscadorVehiculo(\$datoVehiculo)
- insertarRuta(\$datoRuta)
- traerRutasFiltradas(\$filtros)
- cargarVehiculos()

Modelos

adminModel

- getProductos(\$dato_producto)
- actualizarProducto(\$dato_producto)
- actualizarConductor(\$datos_conductor)
- insertarConductor(\$datos_conductor)
- getVehiculos(\$datos_vehiculo)
- actualizarVehiculo(\$datos_vehiculo)
- insertarVehiculo(\$datos_vehiculo)
- getUsuarios(\$datos_usuario)
- actualizarUsuario(\$datos_usuario)
- insertarUsuario(\$datos_usuario)
- getContenedores(\$dato_contenedor)
- actualizarContenedor(\$dato_contenedor)

clientesModel

- `getClientes()`
- `getClienteGeneral($datosCliente)`
- `getClienteEspecifico($datosCliente)`
- `comprobarCliente($datos)`
- `updateCliente($datos)`
- `getClientePorNombre($datoBuscador)`
- `getClientePorDato($datoBuscador)`
- `getClientePorCodigoEESS($datoBuscador)`

contenedorModel

- `getTipo($datos)`
- `setNuevoContenedor($id_tipo, $id_cliente, $datos)`
- `setNuevoEstado($datos)`

direccionModel

- `setNuevaDireccion($id_contenedor, $id_municipio, $datos)`
- `updateDireccion($datos)`

loginModel

- `procesarLogin($datos_login)`
- `traerDatosUsuario($datos_login)`

municipioModel

- `comprobarMunicipio($datos)`

recogidasModel

- `insertarRecogidaHoreca($datosRecogida)`
- `insertarRecogidaContenedor($datosRecogida)`
- `insertarRecogidaEESS($datosRecogida)`
- `traerIdRuta($datosRecogida)`
- `traerRecogidasFiltradas($datos_filtros)`
- `cargarConductores()`
- `cargarProvincias()`

rutasModel

- `getRutasPorDato($datoBuscador)`
- `getConductores($datoConductor)`
- `getVehiculos($datoVehiculo)`
- `cargarVehiculos()`
- `nuevaRuta($datosRuta)`
- `traerRutasFiltradas($datos_filtros)`

3.2.2.3. Cliente

3.2.2.3.1. Diseño de la interfaz



Cientes

Recogidas

Rutas

Estadísticas

Admin

Cerrar Sesión

Panel Administración

Editar Vehiculos

Editar Productos

Editar Zonas

Editar Contenedores

Nuevo Cliente

Editar Conductores

Editar Usuarios

Editar Perfil

Cientes

Recogidas

Rutas

Estadísticas

Admin

Cerrar Sesión

Cientes Sercampo

Cliente

HORECA

CONTENEDOR

EESS

X

PointID	Nombre	CIF	Telefono	CP	Direccion	Municipio	Provincia	Contenedor	Tipo Legal	Activo
ID0000	La Birra de Brian	00000000A	111111111	16510	C/Fronton	Vellisca	Cuenca	Bidón	Horeca	Si
ID0001	Paco Meralgo	11111111B	222222222	16779	La Plaza, 1	Belmontejo	Cuenca	Bidón	Horeca	Si
ID0002	La Tapilla Sixtina	33333333C	333333333	16001	Pl. Mayor	Cuenca	Cuenca	Bidón	Horeca	Si
ID0003	Contenedor Via Publica	99999999S	999999999	16004	Calle Primera	Cuenca	Cuenca	Urbanoil	Contenedor	Si
ID0004	Contenedor Via Publica	99999999S	999999999	28525	Calle Segunda	Rivas	Madrid	Urbanoil	Contenedor	Si
ID0005	Contenedor Via Publica	99999999S	999999999	46005	Calle Tercera	Valencia	Valencia	Urbanoil	Contenedor	Si
ID0006	Gasolinera Por los pelos	33333333C	333333333	16320	KM 420, N-420	Talayuelas	Cuenca	Repsol	EESS Repsol	Si
ID0007	Gasolinera En mitad de la nada	44444444D	444444444	46460	Camino a Silla	Silla	Valencia	Repsol	EESS Repsol	Si
ID0008	EESS Agua fresca de Pepino	55555555D	555555555	45638	Entrada al pueblo	Pepino	Toledo	Repsol	EESS Repsol	Si

58

Cientes

Recogidas

Rutas

Estadísticas

Admin

Cerrar Sesión

La Birra de Brian

PointID	Nombre	CIF	Telefono	CP	Dirección	Municipio	Provincia	Contenedor	Tipo Legal	Activo
ID0000	La Birra de Brian	00000000A	111111111	16510	C/Fronton	Vellisca	Cuenca	Bidón	Horeca	Si

Modificar Cliente

Cientes

Recogidas

Rutas

Estadísticas

Admin

Cerrar Sesión

Modificar La Birra de Brian

Tipo Contenedor

Tipo legal

PointID

Bidón

Horeca

ID0000

Nombre

La Birra de Brian

CIF

00000000A

Telefono

111111111

Dirección

Dirección

C/Fronton

Municipio

Vellisca

Código Postal

16510

Provincia

Cuenca

País

España

Guardar Cliente

Cientes

Recogidas

Rutas

Estadísticas

Admin

Cerrar Sesión

Recogidas

RECOGIDA HORECA

RECOGIDA CONTENEDOR

RECOGIDA EESS

LIMPIAR FORMULARIO

Buscar Horeca

Datos Cliente

Nombre Establecimiento

CIF

Telefono

Dirección

Municipio

Código Postal

Provincia

País

Fecha

dd/mm/aaaa

Conductor

Visitado

Visitado

No visitado

Recogido

Recogido

No Recogido

Litros Recogidos

0

Bidones Recogidos

0

Bidones Entregados

0

Notas

Productos

Desengrasantes

0

Limpiacristales

0

Cerrar Sesión

Cientes

Recogidas

Rutas

Estadísticas

Admin

Cerrar Sesión

Cientes

Recogidas

Rutas

Estadísticas

Admin

Cerrar Sesión

Cientes

Recogidas

Rutas

Estadísticas

Admin

Dirección

Municipio

Código Postal

Provincia

País

Fecha

dd/mm/aaaa

Conductor

Visitado

☒ Visitado

☐ No visitado

Recogido

☒ Recogido

☐ No Recogido

Litros Recogidos

0

Bidones Recogidos

0

Bidones Entregados

0

Notas

Productos

Desengrasantes

0

Fregasuelos

0

Jabón de manos

0

Lavavajillas

0

WC baños

0

Dinero

0

Limpiacristales

0

Lejías

0

Bayetas

0

Higienizantes

0

Filtros

0

Guardar Recogida

Estadísticas

Últimas Recogidas

Últimas Rutas

Últimas Rutas

Rutas a partir de 13/04/2026

2

Rutas totales

1

Recogidas

Asier

Conductor

1

Recogidas

Laura

Conductora

Fecha inicio

dd/mm/aaaa

Fecha final

dd/mm/aaaa

Conductor

Cualquier conductor

Vehículo

Cualquier vehículo

Buscar

Fecha	Conductor	Vehículo	Acciones
2026-05-12	Asier Conductor	Ford Mondeo	Ver detalles
Nombre	Telefono	CIF	Dirección
EESS Agua Fresca De Pepino	55555555	55555555D	Entrada al pueblo, 45638 Pepino
Gasolinera En Mitad De La Nada	44444444	44444444D	Camino a Silla, 46460 Silla
Gasolinera Por Los Pelos	33333333	33333333C	KM 420, N-420, 16320 Talayuelas
2026-05-11	Laura Conductora	VW Polo	Ver detalles
Nombre	Telefono	CIF	Dirección
Contenedor Via Publica	99999999	99999999S	Calle Primera, 16004 Cuenca
La Tapilla Sixtina	33333333	33333333C	Pl. Mayor, 16001 Cuenca

Sercampo

Cerrar Sesión

Cientes

Recogidas

Rutas

Estadísticas

Admin

Últimas Recogidas

Estadísticas a partir de 13/04/2026

430
Litros

1
Recogidas Totales

0
€ entregados

0
Horecas Recogidos

0
Contenedores Recogidos

1
EESS recogidas

Fecha Inicial
dd/mm/aaaa

Fecha Final
dd/mm/aaaa

Conductor
Asier Conductor

Tipo Legal
EESS REPSOL

Provincia
Valencia

Municipio

Buscar

2026-05-12 Gasolinera En mitad de la nada

Más info

TIPO LEGAL
EESS Repsol

LITROS
430

CONDUCTOR
Asier Conductor

MUNICIPIO
Silla

DIRECCIÓN
Camino a Silla

TOTAL €
0

Sercampo

Cerrar Sesión

Cientes

Recogidas

Rutas

Estadísticas

Admin

Últimas Recogidas

Estadísticas a partir de 13/04/2026

915
Litros

5
Recogidas Totales

59
€ entregados

1
Horecas Recogidos

1
Contenedores Recogidos

3
EESS recogidas

Fecha Inicial
dd/mm/aaaa

Fecha Final
dd/mm/aaaa

Conductor
Cualquier conductor

Tipo Legal
Cualquier tipo legal

Provincia
Cualquier provincia

Municipio

Buscar

2026-05-12 Gasolinera En mitad de la nada

Más info

TIPO LEGAL
EESS Repsol

LITROS
430

CONDUCTOR
Asier Conductor

MUNICIPIO
Silla

DIRECCIÓN
Camino a Silla

TOTAL €
0

2026-05-12 EESS Agua fresca de Pepino

Más info

TIPO LEGAL
EESS Repsol

LITROS
235

CONDUCTOR
Asier Conductor

MUNICIPIO
Pepino

DIRECCIÓN
Entrada al pueblo

TOTAL €
0

2026-05-12 Gasolinera Por los pelos

Más info

TIPO LEGAL
EESS Repsol

LITROS
120

CONDUCTOR
Asier Conductor

MUNICIPIO
Talayuelas

DIRECCIÓN
KM 420, N-420

TOTAL €
0

Sercampo

Cerrar Sesión

Cientes

Recogidas

Rutas

Estadísticas

Admin

Vehiculos

Elegir Vehiculo

peu

3333CCC - Peugeot - 9876598765ZZZ

Matrícula: 3333CCC

Modelo: Peugeot

Numero de Bastidor: 9876598765ZZZ

Fecha ITV: 2026-04-16

Fecha Mantenimiento: 2026-05-01

Taller Mantenimiento: Talleres David

Precio Mantenimiento: 650.00

Numero Poliza: POL12222

Empresa Seguro: Reale

Telefono Contacto: 789654789

Telefono Emergencias: 654654654

Editar

Añadir un nuevo vehículo

Datos Vehiculo

Matricula

Modelo

Numero Bastidor

Datos ITV

Fecha ITV

dd/mm/aaaa

Datos taller

Taller Mantenimiento

Precio Mantenimiento

Fecha Mantenimiento

dd/mm/aaaa

Datos Seguro

61



Cientes

Recogidas

Rutas

Estadísticas

Admin

Cerrar Sesión



Cientes

Recogidas

Rutas

Estadísticas

Admin

Cerrar Sesión



Cientes

Recogidas

Rutas

Estadísticas

Admin

Cerrar Sesión

Elegir Vehículo

peu

3333CCC - Peugeot - 9876598765ZZZ

Matrícula: 3333CCC

Modelo: Peugeot

Numero de Bastidor: 9876598765ZZZ

Fecha ITV16/04/2026

Fecha mantenimiento01/05/2026

Taller mantenimientoTalleres David

Precio mantenimiento650.00

Numero PolizaPOLI2222

EmpresaReale

Telefono Contacto789654789

Telefono Emergencias654654654

Guardar

Cancelar

Añadir un nuevo vehículo

Datos Vehículo

Matricula

Modelo

Numero Bastidor

Datos ITV

Fecha ITV

dd/mm/aaaa

Datos taller

Taller Mantenimiento

Precio Mantenimiento

Fecha Mantenimiento

dd/mm/aaaa

Datos Seguro

Numero de Poliza

Empresa del seguro

Telefono Contacto

Telefono Emergencias

Numero de Bastidor: 9876598765ZZZ

Fecha ITV: 2026-04-16

Fecha Mantenimiento: 2026-05-01

Taller Mantenimiento: Talleres David

Precio Mantenimiento: 650.00

Numero Poliza: POLI2222

Empresa Seguro: Reale

Telefono Contacto: 789654789

Telefono Emergencias: 654654654

Editar

Añadir un nuevo vehículo

Datos Vehículo

Matricula

Modelo

Numero Bastidor

Datos ITV

Fecha ITV

dd/mm/aaaa

Datos taller

Taller Mantenimiento

Precio Mantenimiento

Fecha Mantenimiento

dd/mm/aaaa

Datos Seguro

Numero de Poliza

Empresa del seguro

Telefono Contacto

Telefono Emergencias

Guardar Vehículo

Elegir Contenedor

Urbanoil

Interoil

Bidón

Soterrado

Horecoil

Conteroil

Box

Repsol

Interoil

Capacidad100

Notas% *1 (ej 90% -> 90kg)

Editar

62

3.2.2.3.2. Accesibilidad

La accesibilidad web es un aspecto fundamental en el desarrollo de aplicaciones modernas, ya que garantiza que el contenido y las funcionalidades del sistema puedan ser utilizados por el mayor número posible de personas, independientemente de sus capacidades técnicas, cognitivas o sensoriales.

En el desarrollo de la aplicación se han tenido en cuenta diversos principios y pautas de accesibilidad, con el objetivo de ofrecer una interfaz usable y comprensible para todos los perfiles de usuario contemplados en el sistema. Para ello, se han seguido las recomendaciones establecidas en las Pautas de Accesibilidad para el Contenido Web (WCAG 2.1) en su nivel AA, promovidas por el consorcio W3C.

Las principales medidas de accesibilidad implementadas son las siguientes:

- Estructura semántica del contenido: Se ha utilizado HTML5 semántico, empleando etiquetas como <header>, <nav>, <main>, <section> y <footer>, lo que facilita la navegación mediante lectores de pantalla y mejora la interpretación del contenido por parte de tecnologías asistivas.
- Contraste visual adecuado: Se ha cuidado la combinación de colores entre el texto y el fondo, garantizando una relación de contraste suficiente que permita la lectura cómoda incluso a usuarios con visión reducida. La paleta cromática, basada en tonos corporativos verdes y grises, ha sido validada para cumplir con los requisitos mínimos de contraste.
- Etiquetado descriptivo en formularios: Todos los campos de los formularios incluyen etiquetas <label> correctamente asociadas, lo que permite que los lectores de pantalla identifiquen de forma clara cada campo. Asimismo, los mensajes de error y validación se presentan de manera textual y contextual, facilitando su comprensión.
- Indicaciones visuales y textuales: Los mensajes de confirmación, error o advertencia no dependen exclusivamente del color para transmitir información. Se complementan con iconos y textos descriptivos que permiten su interpretación incluso en condiciones de daltonismo o visibilidad reducida.
- Tamaño de fuente y escalabilidad: El diseño de la interfaz utiliza unidades relativas de medida, lo que permite que el texto se escale correctamente en función de las preferencias del usuario y las capacidades del dispositivo, sin pérdida de legibilidad ni desbordamiento de los contenedores.

- **Compatibilidad con distintos dispositivos:** Aunque se aborda con mayor detalle en el diseño responsive, cabe destacar que la aplicación se adapta a diferentes tamaños de pantalla y resoluciones, lo que también contribuye a la accesibilidad, permitiendo el acceso desde dispositivos móviles o con configuraciones de visualización especiales.

En resumen, estas medidas permiten que la aplicación ofrezca una experiencia de uso más inclusiva, cumpliendo con los principios de perceptibilidad, operabilidad, comprensibilidad y robustez que definen la accesibilidad web moderna.

3.2.2.3.3. Usabilidad

La usabilidad es un atributo de calidad que determina la facilidad con la que los usuarios pueden interactuar con una aplicación para alcanzar sus objetivos de manera eficaz, eficiente y satisfactoria. En el caso de nuestra aplicación, se trata de una herramienta de gestión interna destinada a perfiles profesionales con distintos niveles de competencia digital, por lo que la usabilidad ha sido un criterio prioritario durante todo el proceso de diseño y desarrollo, además de ajustarse a las necesidades intrínsecas de la empresa.

Para garantizar una experiencia de usuario óptima, se han aplicado principios de usabilidad basados en la página [W3C](#) y en las buenas prácticas del diseño de interfaces centrado en el usuario. Las decisiones de diseño se han orientado a minimizar los errores y facilitar la curva de aprendizaje.

A continuación, se detallan las principales características de usabilidad implementadas:

- **Interfaz clara y consistente:** La aplicación mantiene una estructura visual y funcional homogénea en todas sus secciones. Elementos como la barra de navegación lateral, los botones de acción, las tablas de datos y los formularios de edición siguen un mismo patrón de diseño, lo que permite al usuario predecir el comportamiento del sistema y desenvolverse con fluidez desde las primeras interacciones.
- **Organización lógica de la información:** La arquitectura de la información se ha diseñado para reflejar las tareas habituales de los usuarios. Las secciones del panel de administración se agrupan por entidades de negocio (clientes, contenedores, rutas, recogidas, etc.), facilitando la localización de cada funcionalidad sin necesidad de navegaciones complejas.

- Retroalimentación constante del sistema: Cada acción del usuario recibe una respuesta clara por parte del sistema. Las operaciones de creación, edición o eliminación de registros se confirman mediante mensajes informativos visibles, y los errores de validación se muestran junto al campo afectado, explicando de forma comprensible el motivo del fallo y cómo solucionarlo.
- Reducción de errores y mecanismos de prevención: Los formularios incluyen validaciones tanto en el lado del cliente como en el servidor. Los campos obligatorios están claramente señalizados y los formatos esperados se indican mediante textos de ayuda o placeholders. En acciones destructivas, como la eliminación de registros, se solicita una confirmación expresa antes de ejecutar la operación, evitando pérdidas de datos accidentales.
- Eficiencia en tareas repetitivas: Las tablas de datos incorporan funcionalidades de búsqueda, filtrado y paginación que facilitan la consulta ágil de grandes volúmenes de información. Los formularios de alta permiten la selección rápida de entidades relacionadas mediante listas desplegables o búsquedas integradas, agilizando las operaciones de registro sin comprometer la precisión de los datos.
- Lenguaje cercano al dominio del usuario: La terminología utilizada en la interfaz se corresponde con el vocabulario propio de la empresa y sus necesidades (contenedores, rutas, recogidas, HORECA, EESS, etc.), evitando tecnicismos innecesarios. Esto facilita la comprensión inmediata de las funcionalidades por parte de los usuarios de la empresa, independientemente de su formación tecnológica previa.
- Diseño responsive: La interfaz se adapta correctamente a distintos tamaños de pantalla (ordenadores de escritorio y tabletas), gracias al uso del framework Bootstrap.
- Persistencia del estado del usuario: La aplicación mantiene la sesión del usuario activa y conserva su posición durante la navegación. El estado de la interfaz (menús desplegados, filtros aplicados o páginas activas) se preserva dentro de la sesión, evitando que el usuario tenga que reconfigurar su entorno de trabajo tras cada recarga o cambio de vista.

En conclusión, el diseño de usabilidad de la aplicación pretende que los usuarios, independientemente de su rol o experiencia técnica, puedan centrarse en las tareas operativas de la empresa sin que la herramienta suponga un obstáculo. La combinación de una interfaz ordenada, una navegación coherente contribuye a una experiencia de

usuario satisfactoria, que se traduce en una mayor productividad y una menor incidencia de errores durante el manejo diario de la aplicación.

3.2.2.3.4. Desarrollo web entorno cliente

A continuación hablamos de cómo hemos trabajado los diferentes aspectos del frontend, realizado con Angular 21, HTML, CSS y Bootstrap.

El archivo styles.css lo hemos utilizado para definir ahí los estilos generales y que engloban a toda la aplicación, por ejemplo la familia de fuentes, el tamaño de la fuente, el tamaño de los títulos, los botones, etc. Utilizando así, los archivos css propios de cada componente para modificar más concretamente cada módulo.

El archivo app.css, se ha usado para cuadrar la estructura de la página y de los componentes Router y Sidebar, asignando el espacio que corresponde a cada uno, y en caso de Login, mostrarlo en toda la página.

a. Formularios y su validación

Se realizan una gran cantidad de formularios en la aplicación, debido a que se deben insertar clientes, contenedores, conductores, vehículos... Estos formularios se realizan almacenando los datos en un objeto, mediante el uso de `[(ngModel)]`, asociando cada valor a un atributo.

Por ejemplo en Conductores-component:

```
<!-- FORMULARIO AÑADIR CONDUCTOR -->
<div class="formulario mt-5">
  <hr>
  <form (ngSubmit)="nuevoConductor(this.conductor_nuevo)">
    <h4>Nuevo Conductor</h4>
    <!-- NOMBRE -->
    <div class="col input-group input-group-sm mb-3">
      <span class="input-group-text">Nombre</span>
      <input
        type="text"
        class="form-control"
        [(ngModel)]="conductor_nuevo.nombre"
        name="nombre_conductor"
        required
      />
    </div>
  </form>
</div>
```

Captura del conductor-component.html

```
conductor_nuevo = {  
  nombre: '',  
  email: '',  
  telefono: '',  
}
```

Captura de conductor-component.ts

En este caso, mediante el uso de ngModel guardamos de forma automática lo que escribe el usuario en el campo, y al darle a enviar, llamamos al método nuevoConductor, pasándole todo el objeto, que mandaremos al back mediante una petición POST.

Es importante, que todo lo manejemos mediante objetos, porque al hacer la petición y envío de datos al back, el backend solo puede leer un JSON a la vez, lo cual resulta en objetos muy grandes en algunas ocasiones.

Se ha intentado refactorizar el código, pero al hacer múltiples peticiones en un solo método nos encontrábamos con problemas de sincronía graves. Las soluciones planteadas fueron, o mandar los objetos aunque fueran muy extensos o el uso de Pipes, y se optó por lo primero, ya que con un control estricto de los nombres tanto en el componente de angular como en el back, es muy sencillo de realizar sin errores.

Ejemplo de diferentes objetos:

```
producto_seleccionado = {  
  id_producto: "",  
  tipo: "",  
  coste: "",  
  notas: ""  
}
```

Captura de editar-producto.component

```
cliente = {  
  id_contenedor: "",  
  id_cliente: "",  
  PointID: "",  
  nombre: "",  
  cif: "",  
  telefono: " ",  
  cod_postal: " ",  
  direccion: " ",  
  municipio: " ",  
  provincia: " ",  
  pais: " ",  
  tipo: " ",  
  tipo_legal: " ",  
  activo: ""  
}
```

Captura de nuevo-cliente.component

```
recogida_nueva = {  
  //datos del contenedor  
  id_contenedor: '',  
  fecha: '',  
  id_ruta: '',  
  id_conductor: '',  
  litros_recogidos: 0,  
  visitado: true,  
  recogida: true,  
  bidones_recogidos: 0,  
  bidones_entregados: 0,  
  notas: '',  
  //datos del producto  
  desengrasante: 0,  
  fregasuelo: 0,  
  lavavajilla: 0,  
  jabon_manos: 0,  
  higienizante: 0,  
  wc_banos: 0,  
  limpia_cristales: 0,  
  lejia: 0,  
  bayeta: 0,  
  filtros: 0,  
  dinero: 0,  
  //datos del cliente  
  nombre: '',  
  cif: '',  
  telefono: '',  
  direccion: '',  
  municipio: '',  
  cod_postal: '',  
  provincia: '',  
  pais: '',  
};
```

Captura de recogidas.component

A la hora de tomar medidas de seguridad en la inserción de datos, en principio, la aplicación será cerrada a la empresa y no se esperan posibles “ataques” a la misma, pero igualmente, además de usar required y tipos de input que correspondan en cada formulario, se comprueban al llamar al método, si los diferentes atributos, que no son accesibles desde el navegador (como puede ser, id_conductor que venía en una consulta previa) está asignado.

```
enviarEdicion(){  
  
  if(this.conductor_seleccionado.id_conductor != '' && this.conductor_seleccionado.nombre != '' && this.conductor_seleccionado.email != '' && this.conductor_seleccionado.telefono != '')(  
    this.adminService.actualizarConductor(this.conductor_seleccionado).subscribe((resultado: any) => {  
      //datos del cliente  
      nombre: '',  
      cif: '',  
      telefono: '',  
      direccion: '',  
      municipio: '',  
      cod_postal: '',  
      provincia: '',  
      pais: '',  
    }  
  )  
}
```

En la captura anterior, se verifica si `id_conductor` es `!=` de `" "`, siendo un dato ajeno al formulario que no sale del componente, por lo que no puede alterarse de forma ajena.

Además de todo eso, en el backend, se realizan consultas preparadas de PHP mediante los métodos de PDO. Evitando así posibles inserciones de código a la base de datos.

```
public function insertarVehiculo($datos_vehiculo){
    try {

        $this->db->beginTransaction();

        $sqlInsertarSeguro = "INSERT INTO seguros_vehiculos (num_poliza, tel_emergencias, tel_contacto, empresa)
                                VALUES (:num_poliza, :tel_emergencias, :tel_contacto, :empresa)";

        $stmt = $this->db->prepare($sqlInsertarSeguro);

        $stmt->bindParam(':num_poliza', $datos_vehiculo['num_poliza'], PDO::PARAM_STR);
        $stmt->bindParam(':tel_emergencias', $datos_vehiculo['tel_emergencias'], PDO::PARAM_STR);
        $stmt->bindParam(':tel_contacto', $datos_vehiculo['tel_contacto'], PDO::PARAM_STR);
        $stmt->bindParam(':empresa', $datos_vehiculo['empresa'], PDO::PARAM_STR);

        $stmt->execute();
    }
}
```

b. Manejo y gestión de eventos: Teclado, ratón y estados de la ventana

Eventos que tenemos en cuenta en nuestra aplicación son, click y escribir en el teclado, y en algunos formularios, que realice un método al enviar el formulario.

Los controles de click que usamos son:

- Para redirigir, al clicar en diferentes botones y lugares de la aplicación, llamamos a un método que usa, `"this.router.navigate(['confirmado']);"`, importando el módulo Router al componente correspondiente. Este sistema lo usamos, en los casos donde queremos realizar acciones previas a redirigir, como puede ser, hacer peticiones al back. (Por ejemplo, para redirigir a un `id_cliente` específico)
- Para redirigir, al clicar en un botón (y exclusivamente en botones), se puede usar de forma alternativa el método `[routerLink]="['/recogidas']"`, importando el módulo RouterModule, distinto al anterior, que nos permite ir a la dirección escrita en el corchete sin necesidad de añadir código extra al componente.
- Otro de los eventos que se producen al hacer click en diferentes elementos, son los que muestran datos y despliegan diferentes tablas o listas en base a información traída previamente del back y almacenada en objetos o arrays.

Como es el ejemplo del botón *Más Info* en el apartado de últimas recogidas:

```

<div class="ult-recogidas-container">
  @for (recogida of array_recogidas; track $index) {
    <div class="card mb-4 custom-card">
      <div class="card-body">
        <div class="title-boton">
          <h5 class="card-title">{{ recogida.fecha }} - {{ recogida.nombre_cliente }}</h5>

          <div class="boton-masinfo mb-3">
            <button
              class="btn btn-sercampo btn-sm"
              type="button"
              data-bs-toggle="collapse"
              [attr.data-bs-target]="'#collapse' + $index"
              aria-expanded="false"
            >
              Más info
            </button>
          </div>
        </div>
      </div>
    </div>
  }
</div>

```

Donde llamamos a div con `id = "collapse{{ $index }}"` y mostramos la información almacenada en `recogida`, dentro del `array_recogidas`, cargado previamente.

- El elemento (*keyup*), para recoger cuando se está usando el teclado dentro de un input, lo usamos en la mayoría de las páginas donde queremos realizar una búsqueda y no sabemos la cantidad de resultados que podemos obtener o es demasiado extenso. La forma en la que lo hemos utilizado es mandando el interior del input deseado al back, para obtener la información similar (LIKE) y que cumpla los diferentes filtros.

```

$sql = "SELECT con.id_contenedor, c.id_cliente, c.PointID, c.nombre, c.cif, c.telefono, d.cod_postal, d.direccion,
m.municipio, m.provincia, m.pais, t.tipo, con.tipo_legal, con.activo, cod.cod_eess,
(SELECT MAX(rec.fecha)
FROM recogidas AS rec
WHERE rec.id_contenedor = con.id_contenedor
AND rec.fecha <= CURRENT_DATE) AS ultima_fecha
FROM contenedores AS con
INNER JOIN clientes AS c ON con.id_cliente = c.id_cliente
INNER JOIN tipo_contenedor AS t ON con.id_tipo_contenedor = t.id_tipo_contenedor
INNER JOIN direcciones AS d ON con.id_contenedor = d.id_contenedor
INNER JOIN municipios AS m ON d.id_municipio = m.id_municipio
LEFT JOIN codigos_eess AS cod ON c.id_cliente = cod.id_cliente
WHERE (m.municipio LIKE :datoBuscador OR c.nombre LIKE :datoBuscador OR cod.cod_eess LIKE :datoBuscador) AND (con.activo = true)
ORDER BY m.provincia, m.municipio";

$stmt = $this->db->prepare($sql);

$dato = "% " . $datoBuscador['dato'] . "%";

$stmt->bindParam(':datoBuscador', $dato, PDO::PARAM_STR);

```

En la captura anterior, traemos un valor `$datoBuscador['dato']` que corresponde a lo que se está escribiendo en el input, y buscamos cualquier resultado que corresponda, ya sea municipio, nombre o un código.

Para limitar el cambio brusco del DOM, hemos limitado la búsqueda en la gran mayoría de buscadores a mínimo tres caracteres, ya que si no, llegaban demasiados datos con una sola búsqueda.

```
buscarGeneral() {
  if (this.dato_buscado.dato.length >= 3) {
    this.rutasService.traerDatosBuscador(this.dato_buscado).subscribe((resultado: any) => {
      this.array_buscador = resultado; //Esto es el array
      this.cdr.detectChanges();
      console.log(resultado);
    });
  } else {
    this.array_buscador = [];
  }
}
```

Y lo mostramos de la siguiente manera en el HTML:

```
<input
  type="text"
  class="form-control buscador_Buscador"
  name="nombre_buscado"
  placeholder="Buscar Lugar"
  [(ngModel)]="dato_buscado.dato"
  (keyup)="buscarGeneral()"
/>
</div>
<div class="col-1 icono-vaciar">
  <i class="bi bi-x-square icono" (click)="vaciarBuscador()"></i>
</div>
</div>
</div>
<div class="resultados">
  <!-- Resultados -->
  <ul class="list-group">
    @for (cliente_encontrado of array_buscador; track cliente_encontrado.id_cliente) {
      <li class="list-group-item buscador_item" (click)="meterEnRuta(cliente_encontrado)">
        {{cliente_encontrado.cod_eess}} {{ cliente_encontrado.nombre }}, {{ cliente_encontrado.direccion }},
        {{ cliente_encontrado.municipio }},
        {{ cliente_encontrado.provincia }} ---- Ultima Recogida: <b>{{cliente_encontrado.ultima_fecha}}</b>
      </li>
    }
  </ul>
```

Mostrando los datos que queramos de todos los que consultamos en el modelo.

c. Gestión y almacenamiento de datos e información en el cliente

En el cliente, hemos usado el localStorage, para almacenar la información de la sesión, así evitamos tener que iniciar sesión cada vez que recargamos la aplicación y nos permite comprobar el rol del usuario de forma rápida y desde un solo servicio para limitar las direcciones donde puede acceder. Se ha tenido que importar *PLATFORM_ID*, para acceder al token del navegador, y *isPlatformBrowser*, por seguridad de la misma.

Almacenamos concretamente un objeto de tipo Usuario, que tiene la siguiente estructura:

```
export interface Usuario {  
  id_usuario: number;  
  nombre: string;  
  email: string;  
  telefono: string;  
  rol: string;  
}
```

Usamos el método cerrarSesion() de login-service cada vez que, clickamos el botón correspondiente en el sidebar, llegamos a una ruta que no corresponde a la asignada para nuestro usuario de forma externa o falta algún dato en el objeto almacenado, para garantizar así la seguridad de la aplicación.

```
cerrarSesion() {  
  //Pone a false y a null los atributos.  
  this.EstadoLogin.set(false);  
  this.UsuarioActual.set(null);  
  //Elimina los items/objetos almacenados en localStorage.  
  if (isPlatformBrowser(this.platformId)) {  
    localStorage.removeItem('sesionActiva');  
    localStorage.removeItem('usuario');  
  }  
}
```

d. Modificación del DOM

Gracias a estar usando la última versión de Angular, la 21, las modificaciones del DOM se han realizado de forma sencilla y automática al usar los métodos @if y @for en el HTML, además, en teoría no sería necesario usar el módulo *ChangeDetectorRef* por la misma razón, pero el funcionamiento de la aplicación mejora usándolo.

Eso, sumado al uso de *[[ngModel]]* y el uso de corchetes *{{}}* para referenciar a elementos del componente, ha sido todo lo necesario para los cambios en el DOM.

e. Animaciones, efectos, cambios dinámicos de estilos etc... para dinamizar la parte visible al cliente

Las animaciones que tenemos en la aplicación se basan todas en acciones del usuario, a excepción del login, que ocurre al cargar la página.

Ejemplo de la animación del login utilizando “@keyframes”:

```
/* Animación suave de entrada */
@keyframes fadeInUp {
  from {
    opacity: 0;
    transform: translateY(30px);
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}
```

El resto de animaciones de Hover o de Active, se realizan tanto con pseudo-elementos como con pseudo-clases y han consistido en cambiar el color de fondo, el color de la tipografía o la sombra en la mayoría de los casos.

```
.btn-contenedor:hover{
  box-shadow: 0 3px 7px #4d5b9fa5
}
```

```
/* Para switch de Bootstrap */
.form-switch .form-check-input:checked {
  background-color: #28a745; /* Verde */
  border-color: #28a745;
}
```

También se han definido variables y clases de Bootstrap, con distintos efectos en el archivo Styles.css para que afecten a todo el documento con tan solo poner la clase correspondiente, o poder usar las variables en un css específico de un componente.

```
.icono::before {
  content: "\F629";
  font-size: 4vh;
  color: var(--bs-danger);
}

.icono:hover::before {
  content: "\F628"; /* código del icono hover */
  font-size: 4vh;
  color: var(--bs-danger);
  cursor: pointer;
}
```

Usando siempre las clases de bootstrap y siendo accesibles desde styles.css a todo el proyecto, podíamos mantener el mismo patrón de diseño en base a lo definido en la guía estilos, con los colores corporativos de Sercampo y su tipografía.

```

/* BOTONES */
.btn-sercampo {
  border: none;
  background-color: #4d5b9f;
  color: white;
}
.btn-sercampo:hover{
  background-color: #ff8f50;
  color: white;
  box-shadow: 0 1px 5px rgba(0, 0, 0, 0.3);
}

.btn-filtro{
  background-color: #4d5b9f;
  color: white;
}

.btn-filtro:hover{
  background-color: white;
  color: #4d5b9f;
  border: 1px solid #4d5b9f;
}
.btn-filtro.active{
  background-color: #ff90507c;
  color: black;
  border: white;
}

```

f. Comunicación AJAX

Para la comunicación AJAX, hemos realizado una serie de servicios que son los que realizan la petición al back. Primero, en el componente nos suscribimos al método que tenemos en el servicio, al suscribirnos ese método espera una respuesta.

En el propio servicio es donde hacemos la petición `httpClient`, a la API de nuestro back. Además definimos aquí las cabeceras que queremos que lleve esa petición.

En el back, en el archivo `index.php`, es donde definimos las cabeceras de nuevo, en principio no sería necesario poner las que hemos puesto al hacer la petición, pero nos evitamos posibles errores poniéndolas en los dos lugares.

La API del back siempre devuelve un archivo JSON por como está definido, y el servicio lo “recoge” y se lo da como respuesta al método del componente.

En principio se usó el método `httpClient POST` en todos los métodos y comunicaciones con el back, ya que servía tanto para recibir como para mandar información, independientemente de si se enviaba con datos o no. Esto fue porque de forma errónea entendimos que era algo aceptado, pero se saltaba los estándares establecidos y las buenas prácticas y se corrigió usando los métodos `GET`, `PUT`, `DELETE` y `PATCH`.

A excepción de obtenerRutas y de obtenerRecogidas, que no lo procesaba correctamente el método GET, ya que el uso que se hacía del JSON era demasiado “complejo” para este método. Tras investigar y ver prácticas aceptadas, entendimos que cuando son filtros o datos más complejos, el método GET no es suficiente, y está aceptado usar el método POST para esa complejidad extra.

Lo único distinto en estos dos métodos respecto a lo que se hacía en los demás es concatenar código SQL a la consulta en función de si hay datos asignados a distintas variables/atributos del JSON.

```
1 reference | 0 overrides
public function traerRecogidasFiltradas($datos_filtros){

    try {

        $this->db->beginTransaction();

        $sql = "SELECT c.nombre AS nombre_cliente, con.tipo_legal, rec.litros_recogidos, rec.fecha, conduc.nombre AS nombre_conductor, m.municipio, m.provincia, d.direccion,
        rec.id_recogida, rec.id_ruta, rut.id_conductor, con.id_contenedor, c.id_cliente, 0 AS total_intercambio
        FROM recogidas AS rec
        INNER JOIN rutas AS rut ON rec.id_ruta = rut.id_ruta
        INNER JOIN conductores AS conduc ON conduc.id_conductor = rut.id_conductor
        INNER JOIN contenedores AS con ON rec.id_contenedor = con.id_contenedor
        INNER JOIN clientes AS c ON con.id_cliente = c.id_cliente
        INNER JOIN direcciones AS d ON con.id_contenedor = d.id_contenedor
        INNER JOIN municipios AS m ON d.id_municipio = m.id_municipio
        WHERE (rec.fecha >= :fechaInicial) AND (rec.fecha <= :fechaFinal)";

        if ($datos_filtros['conductor'] != "") {
            $sql .= " AND (rut.id_conductor = :id_conductor)";
        }

        if ($datos_filtros['tipo_legal'] != "") {
            $sql .= " AND (con.tipo_legal = :tipo_legal)";
        }

        if ($datos_filtros['municipio'] != "") {
            $sql .= " AND (m.municipio LIKE :municipio)";
        }

        if ($datos_filtros['provincia'] != "") {
            $sql .= " AND (m.provincia LIKE :provincia)";
        }

        $sql .= " ORDER BY rec.fecha DESC";
    }
}
```

```

$stmt->bindParam(':fechaInicial', $datos_filtros['fechaInicial'], PDO::PARAM_STR);
$stmt->bindParam(':fechaFinal', $datos_filtros['fechaFinal'], PDO::PARAM_STR);

if ($datos_filtros['conductor'] != "") {
    $stmt->bindParam(':id_conductor', $datos_filtros['conductor'], PDO::PARAM_STR);
}

if ($datos_filtros['tipo_legal'] != "") {
    $stmt->bindParam(':tipo_legal', $datos_filtros['tipo_legal'], PDO::PARAM_STR);
}

if ($datos_filtros['municipio'] != "") {
    $municipio = "%" . $datos_filtros['municipio'] . "%";
    $stmt->bindParam(':municipio', $municipio, PDO::PARAM_STR);
}

if ($datos_filtros['provincia'] != "") {
    $provincia = "%" . $datos_filtros['provincia'] . "%";
    $stmt->bindParam(':provincia', $provincia, PDO::PARAM_STR);
}

```

Al ser el único lugar donde fallaba el método GET, y funcionar a la perfección el sistema de filtros, decidimos mantener el uso de POST en estos dos casos, ya que es correcto usarlos cuando es complejo el JSON y además aportan seguridad extra al sistema.

g. Comunicación asíncrona con el servidor

Esos métodos que llamamos de los servicios son del tipo observable, para ello hemos importado el módulo Observable dentro de la librería rxjs, incluida con angular. Mediante el uso de este módulo, hemos evitado problemas de sincronía del componente del front respecto al back, ya que podemos usar los “métodos” *next*, *error* y *complete* dentro del subscribe. Permitiéndonos esos “métodos” actualizar de forma dinámica un array o un objeto que mostramos posteriormente en el HTML o que usamos en otros métodos.

3.2.3. Fase de despliegue

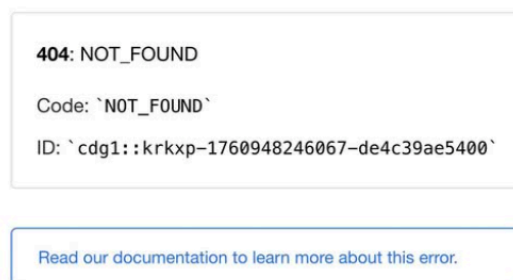
3.2.3.1. Despliegue utilizando un hosting

Aunque la aplicación se ha desarrollado para desplegarse de forma autónoma dentro de un equipo/servidor dentro de la red de sercampo, ha sido desplegada en un dominio público con datos inventados para la exposición de este proyecto en el centro.

Se siguieron una serie de pasos, y se sortearon distintos errores que se exponen a continuación para lograr el correcto funcionamiento de la aplicación y su correcta comunicación con la API propia del back y la base de datos correspondiente.

Se comenzó desplegando el frontend, realizado con Angular, en Vercel, una web que proporciona dominios gratuitos y centrada en el uso de diferentes frameworks como Angular, entre otros, siendo muy sencilla de implementar ya que con la propia conexión desde tu cuenta en github puedes importar el proyecto y elegir una rama en el repositorio de GitHub y compila de forma automática en cuanto detecta cambios en la misma.

Aquí nos surgió un problema con la forma de compilar el proyecto, ya que teníamos rutas variables y ng build no podía compilar las rutas sin definir.



Tras distintas investigaciones y tutoriales, descubrimos que había que definir el tipo de render que tenían esas rutas, y en el archivo *app.routes.server.ts* añadir lo siguiente.

```
frontend > src > app > app.routes.server.ts > ...
1 import { RenderMode, ServerRoute } from '@angular/ssr';
2
3 export const serverRoutes: ServerRoute[] = [
4   {
5     path: '**',
6     renderMode: RenderMode.Prerender
7   },
8   {
9     path: 'cliente/:id_cliente',
10    renderMode: RenderMode.Server
11  },
12  {
13    path: 'confirmado/:id_cliente',
14    renderMode: RenderMode.Server
15  },
16  {
17    path: 'modificar/:id_cliente',
18    renderMode: RenderMode.Server
19  }
20 ];
```

Cuando se tenía desplegado el frontend, y ya pudiendo visualizar desde el navegador la página de login, fuimos a desplegar el backend, y siguiendo los mismos pasos, nos encontramos con un

error extraño, en lugar de cargar lo esperado en el backend (controlador no encontrado), nos descargaba directamente del navegador un archivo index, que contenía el código de nuestro index.php.

```
61
62 if (file_exists($controllerFile)) {
63     require_once $controllerFile;
64     $controllerInstance = new $controllerName();
65
66     // Ejecutar acción
67     if (method_exists($controllerInstance, $action)) {
68         // Pasar los parámetros al método
69         $controllerInstance->$action($parametros);
70     } else {
71         http_response_code(404);
72         echo json_encode(['error' => "Acción no encontrada"]);
73     }
74 } else {
75     http_response_code(404);
76     echo json_encode(['error' => "Controlador no encontrado"]);
77 }
78 ...>
```



Tras distintas pruebas y fallos, descubrimos que Vercel no es compatible con php vanilla, por lo que tuvimos que dejar apartada la idea de usar Vercel por el momento y mudamos el proyecto a nuestra segunda opción.

InfinityFree es una página, de nuevo gratuita, con una interfaz muy sencilla, entendimos rápidamente lo que ocurría y nos permitió realizar distintas pruebas.

La inserción a la base de datos que proporciona InfinityFree fue muy rápida, ya que usa phpMyAdmin como gestor de base de datos, al igual que XAMPP donde habíamos desarrollado el proyecto, y nos permitió desplegar el backend hasta dar con el error esperado (controlador no encontrado).

Decidimos en ese momento que desplegaríamos el frontend también en InfinityFree para tener el proyecto al completo en una sola URL, suponiendo que sería más fácil de comunicar el frontend con el backend.

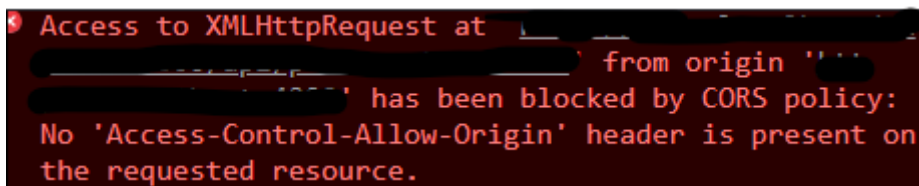
Tras compilar en nuestros equipos el proyecto, ya que InfinityFree no entiende el proyecto de Angular sin la compilación previa, nos encontramos con nuevos errores relativos al index.html de Angular.

Tras multitud de pruebas haciendo cambios en el archivo .htaccess, y distintas formas de afrontar los problemas, descubrimos que el hosting InfinityFree no es compatible con los proyectos SSR (Server Side Rendering) de Angular, ya que no permite el render en el lado del servidor. Tras intentar variar la configuración del proyecto para que sea CSR (Client Side Rendering) y se renderice en el lado del cliente, descubrimos que no es posible realizar este cambio, y se debería haber seleccionado este tipo de

renderizado a la hora de crear el proyecto, pero dado que elegimos SSR, teníamos que descartar Infinity Free para el despliegue del Frontend.

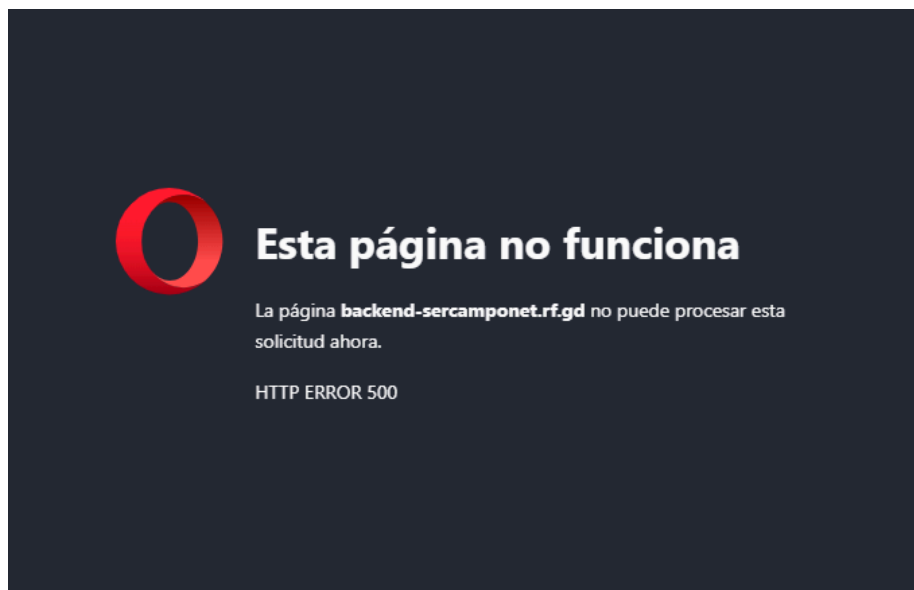
Al tener el backend en InfinityFree y el Frontend funcional en Vercel, decidimos modificar los archivos subidos a cada uno para que permitiesen la comunicación entre ellos, y funcionase Angular en Vercel, haciendo peticiones a nuestro dominio de InfinityFree.

Tras realizar todos esos cambios, y lo que creíamos que debería funcionar, nos encontrábamos en InfinityFree con un error 500 del navegador y en Vercel con un fallo del CORS.



```
Access to XMLHttpRequest at [redacted] from origin '[redacted]' has been blocked by CORS policy: No 'Access-Control-Allow-Origin' header is present on the requested resource.
```

El fallo del CORS está relacionado con las cabeceras y los permisos del backend, un fallo que nos habíamos encontrado anteriormente en el desarrollo, y tras distintos cambios, se llegó a la conclusión de que el error era exclusivamente del backend y estaría relacionado con ese error 500.



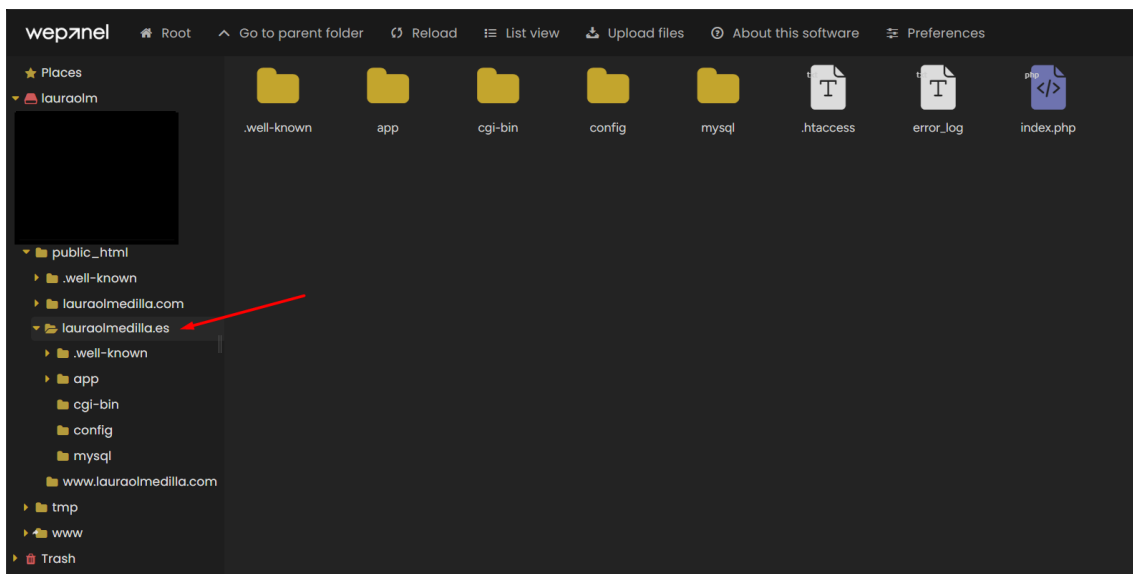
Siendo este un error genérico y sin más información al respecto del fallo, tuvimos que investigar en los distintos blogs, videos y documentación de ambas páginas para descubrir por qué ocurría dicho error.

Descubrimos que el hosting de InfinityFree no permite usar de la manera que nosotros lo hacíamos sus servicios, ya que no permiten el alojamiento de sitios que utilicen la lógica de peticiones por POST, GET, PUT..., siendo estas bloqueadas automáticamente

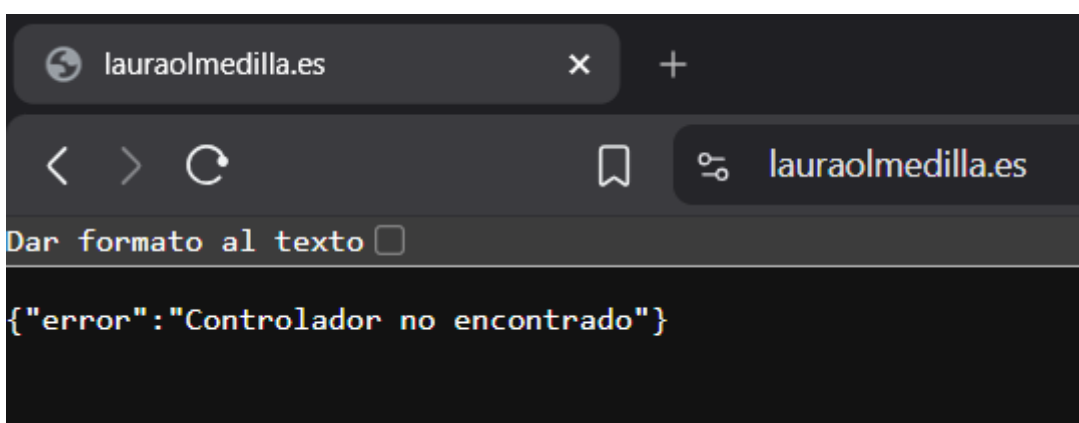
para evitar bots y spam, al haber realizado una API de la forma en la que lo hemos hecho.

Tras consultar distintas opciones disponibles y cómo podíamos afrontar el problema, se decidió probar usando el dominio propio de *lauraolmedilla.es*, siendo esta la última opción por ser algo ajeno al proyecto y un dominio personal de Laura utilizado para su proyecto profesional.

Se traspasó de nuevo la base de datos y el backend a dicho dominio, se configuró para poder acceder a dicha base de datos y el .htaccess de la manera que correspondía para que funcione.



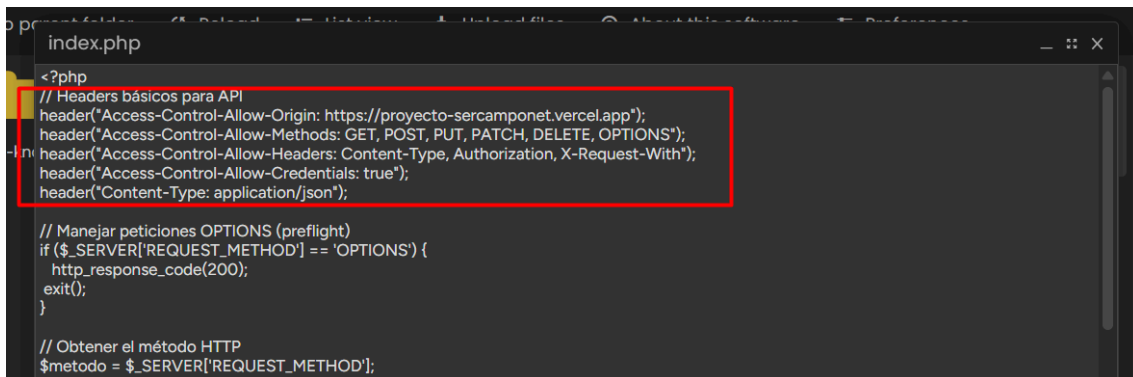
Dando el error esperado de Controlador No Encontrado, y funcionando correctamente como una API.



Se cambió de nuevo el frontend para que se comuniqué correctamente con el backend.

```
private url = 'https://lauraolmedilla.es/index.php';
```


Y las cabeceras de index.php del backend para que permitan exclusivamente peticiones desde nuestra web desplegada.



```
index.php
<?php
// Headers básicos para API
header("Access-Control-Allow-Origin: https://proyecto-sercamponet.vercel.app");
header("Access-Control-Allow-Methods: GET, POST, PUT, PATCH, DELETE, OPTIONS");
header("Access-Control-Allow-Headers: Content-Type, Authorization, X-Request-With");
header("Access-Control-Allow-Credentials: true");
header("Content-Type: application/json");

// Manejar peticiones OPTIONS (preflight)
if ($_SERVER['REQUEST_METHOD'] == 'OPTIONS') {
    http_response_code(200);
    exit();
}

// Obtener el método HTTP
$metodo = $_SERVER['REQUEST_METHOD'];
```

Se corrigieron errores nuevos en el funcionamiento de la aplicación que habían surgido al hacer la compilación, ya que algunos tipos de los atributos de typescript cambian al compilarse y pueden dar errores.

Finalmente se desplegó correctamente la página y ya es visible de forma pública en el enlace indicado.

3.2.3.2. Despliegue en un equipo local propio

Para desplegar el proyecto en un equipo local, se ha utilizado XAMPP para la parte de backend y el comando ng serve, dentro de npm de Angular para la parte de frontend. XAMPP incluye el gestor de base de datos, phpmyadmin que ha sido bastante cómodo de utilizar, ya que es lo que habíamos visto en clase durante este curso, permitiéndonos crear una base de datos lista para el despliegue.

XAMPP Apache hace las funciones de API, siendo su ruta localhost puerto 80 y 443, se pone dicha ruta en la configuración del frontend, para que sea a dicha dirección donde hagan las peticiones.

Angular se ejecuta en la ruta localhost puerto 4200, siendo esta la ruta permitida en la cabecera de index.php en el backend.

Una vez ejecutados ambos, la comunicación era prácticamente instantánea permitiendo un desarrollo fluido y fácilmente debuggable.

Para el despliegue local en sercampo se ha utilizado un servidor con debian donde desplegar el frontend en una ip fija dentro de la red de sercampo y el backend ejecutándose dentro de docker en el mismo equipo.

3.3. Seguimiento y control de incidencias

Durante el proceso de desarrollo del proyecto, se ha establecido un sistema estructurado para el seguimiento y control de incidencias, con el objetivo de identificar, registrar, analizar y resolver de forma sistemática cualquier anomalía, error o desviación funcional detectada en la aplicación.

Este sistema ha permitido mantener la trazabilidad de los problemas y garantizar que las correcciones se implementarán de manera ordenada y eficiente.

Para la gestión de incidencias se ha adoptado un flujo de trabajo basado en las siguientes etapas:

1. Identificación y registro: Toda incidencia detectada durante las fases de desarrollo, integración o pruebas ha sido registrada en una base de datos propia creada en la herramienta Notion, adaptada específicamente al flujo de trabajo del proyecto. Para cada incidencia se han documentado al menos los siguientes campos: título descriptivo, fecha de detección, descripción detallada del problema, área afectada, prioridad asignada, persona responsable de la resolución y fecha límite prevista.
2. Análisis y priorización: Cada incidencia ha sido evaluada para determinar su causa raíz y su impacto sobre el funcionamiento del sistema. A partir de dicho análisis, se ha asignado una etiqueta de prioridad (alta, media o baja), de modo que los errores de prioridad alta (aquellos que impedían el correcto funcionamiento de una funcionalidad principal) fueran atendidos inmediatamente, mientras que las incidencias de prioridad media o baja se programaban en momentos posteriores o se reservaban como mejoras potenciales.
3. Asignación y corrección: Las incidencias han sido asignadas al miembro del equipo con más tiempo disponible en ese momento o el que había realizado los cambios que provocaron dicho error, quedando reflejada dicha asignación en la base de datos de Notion para garantizar la responsabilidad y el seguimiento. Una vez corregidas, se ha procedido a verificar la solución mediante pruebas específicas, asegurando que la corrección no introdujera efectos no deseados en otras partes del sistema.
4. Verificación y cierre: Tras la corrección, se ha realizado una validación adicional para confirmar que la incidencia ha quedado completamente resuelta, normalmente siendo verificado tanto por Laura como por Asier en sus

respectivos equipos. En caso afirmativo, se ha procedido al cierre del registro en Notion, documentando la solución aplicada y la fecha de resolución.

Notion es una plataforma colaborativa que ha permitido crear una base de datos totalmente personalizada y adaptada a las necesidades específicas del proyecto. Las principales funcionalidades empleadas han sido:

1. Base de datos de tareas: Se ha diseñado una base de datos con propiedades personalizadas (fechas, selectores, textos, checkboxes) que ha facilitado la estandarización del registro y la consulta ágil de la información.
2. Etiquetas de prioridad: Se han definido tres niveles de prioridad (alta, media y baja) que han permitido clasificar las incidencias según su urgencia y necesidad, facilitando así la planificación de las tareas de corrección.
3. Etiquetas por área: Se ha implementado un sistema de etiquetado por áreas funcionales o tecnológicas, lo que ha permitido filtrar y agrupar las incidencias según el módulo del sistema al que pertenecían. Las principales áreas definidas han sido:
 - BBDD: Incidencias relacionadas con el diseño, las consultas o la integridad de la base de datos, la API REST o los controladores
 - Angular: Problemas en el frontend, como errores en componentes, servicios o enrutamiento.
 - Bootstrap/CSS: Incidencias vinculadas a la maquetación, el diseño responsive o los estilos visuales.
 - Documentación: Correcciones o mejoras en la memoria, manuales o comentarios del código.
 - Sercampo: Errores ajenos a nuestras responsabilidades o competencias como becarios.
4. Asignación de tareas: Cada incidencia ha sido asignada a un responsable concreto, lo que ha permitido distribuir la carga de trabajo de manera equitativa y garantizar que cada problema tuviera un propietario claro para su resolución.
5. Fechas y seguimiento: Se han registrado tanto la fecha de detección como la fecha de resolución, permitiendo calcular tiempos de respuesta y elaborar informes de evolución del proyecto.

mostrar modal al enviar ruta,  ABRIR	● Listo	23 de abril de 2026	Media	Bootstrap	23 de abril de 2026 12:11	 Laura
Centrar botones de debajo de los formularios	● Listo	24 de abril de 2026	Baja	Bootstrap design	22 de abril de 2026 13:52	A Asier Martínez
si se mete mal un nuevo cliente, que avise	● Listo	24 de abril de 2026	Alta	BBDD Angular Bootstrap	22 de abril de 2026 10:02	A Asier Martínez
Hacer estadísticas	● Listo	11 de mayo de 2026 →	Media	BBDD Angular Bootstrap	22 de abril de 2026 8:56	A Asier Martínez
Editar formulario modificar-cliente para c	● Listo	11 de mayo de 2026 →	Media	Bootstrap design	22 de abril de 2026 8:55	 Laura

Como resultado de la aplicación sistemática de este proceso de seguimiento y control mediante Notion, se ha logrado:

Mantener un registro histórico completo y accesible de todos los problemas encontrados y las soluciones aplicadas, lo que ha facilitado la labor de depuración y ha evitado la recurrencia de errores ya corregidos.

Mejorar la comunicación y coordinación, al disponer de una base de datos centralizada y actualizada en tiempo real sobre el estado de las tareas pendientes.

Visualizar de forma clara la distribución del trabajo por áreas tecnológicas, lo que ha permitido identificar los módulos que han requerido mayores esfuerzos de corrección.

En conclusión, el sistema de seguimiento y control de incidencias basado en Notion ha resultado fundamental para garantizar la fiabilidad de la aplicación desarrollada, permitiendo una respuesta ágil ante los problemas y contribuyendo a la consecución de los objetivos de calidad definidos en el proyecto.

3.4. Indicadores de calidad de procesos

Durante el desarrollo del proyecto, la calidad de los procesos se ha garantizado mediante la aplicación de buenas prácticas de desarrollo y una verificación continua del producto. Aunque no se han definido indicadores cuantitativos formales, se han utilizado criterios cualitativos como la cobertura total de los requisitos funcionales, la resolución sistemática de las incidencias registradas en Notion (apartado 3.3) y el cumplimiento de la guía de estilo establecida.

4. Recursos materiales

Para el desarrollo del presente proyecto, se han empleado diversos recursos materiales, tanto hardware como software, que han resultado imprescindibles para llevar a cabo las tareas de análisis, diseño, implementación, pruebas y documentación. A continuación, se detallan los principales medios utilizados, así como su valoración económica.

<i>Concepto</i>	<i>Características</i>			<i>Unidades</i>	<i>Coste unitario</i>	<i>Coste total</i>
PC 1	Procesador	Intel		1	800,00 €	800,00 €
	Core i7, 16 GB RAM, SSD 512 GB					
PC 2	Procesador	Intel		1	600,00 €	600,00 €
	Core i5, 8 GB RAM, SSD 256 GB					
Monitor externo	Pantalla 24" Full HD			1	150,00 €	150,00 €
Ratón y teclado	Inalámbricos			2 pares	30,00 €	60,00 €
Conexión Internet	a	Fibra óptica	300 Mbps	2 líneas	30,00 €/mes	60,00 € *

Total hardware: 1.670,00 €

Para el desarrollo y gestión del proyecto se han utilizado las siguientes herramientas software, en su mayoría de carácter gratuito u open source, lo que ha permitido reducir significativamente los costes del proyecto:

<i>Concepto</i>	<i>Versión</i>	<i>Licencia</i>	<i>Coste</i>
Visual Studio Code	Última estable	Gratuita (open source)	0,00 €
XAMPP (Apache, MySQL, PHP)	8.x	Gratuita (open source)	0,00 €
Angular CLI	21	Gratuita (open source)	0,00 €
Node.js	Última LTS	Gratuita (open source)	0,00 €
Git	Última estable	Gratuita (open source)	0,00 €
GitHub	-	Gratuita	0,00 €
Notion	-	Gratuita (plan personal)	0,00 €
MySQL Workbench	-	Gratuita	0,00 €
Postman	-	Gratuita	0,00 €
Bootstrap	5.x	Gratuita (open source)	0,00 €

Figma	-	Gratuita	0,00 €
Enterprise Architect	9	Licencia educativa	0,00 €
Microsoft Office / Google Docs	-	Gratuita / Educativa	0,00 €
Sistema operativo (Windows/Linux)	-	Licencia personal	0,00 €
Excalidraw	-	Gratuita	0.00€
Draw.io	-	Gratuita	0.00€

Total software: 0,00 €

La elección de herramientas de código abierto y gratuitas ha permitido mantener el coste del proyecto en niveles reducidos, demostrando que es posible desarrollar una aplicación web profesional sin necesidad de realizar grandes inversiones iniciales en licencias de software. Este aspecto resulta especialmente relevante para pequeñas y medianas empresas que deseen implantar soluciones tecnológicas con presupuestos ajustados.

4.1. Inventario, valorado de medios

A continuación, se detalla el inventario de los medios materiales utilizados durante el desarrollo del proyecto, junto con su correspondiente valoración económica.

Concepto	Características	Unidades	Coste unitario	Coste total
PC 1	Intel Core i7, 16 GB RAM, SSD 512 GB	1	800,00 €	800,00 €
PC 2	Intel Core i5, 8 GB RAM, SSD 256 GB	1	600,00 €	600,00 €
Visual Studio Code	Editor de código	1	0,00 €	0,00 €
XAMPP	Servidor local (Apache, MySQL, PHP)	1	0,00 €	0,00 €
Angular CLI	Framework frontend	1	0,00 €	0,00 €
Node.js	Entorno JavaScript	1	0,00 €	0,00 €

Git / GitHub	Control de versiones	1	0,00 €	0,00 €
Notion	Gestión de incidencias	1	0,00 €	0,00 €
Bootstrap 5	Framework CSS	1	0,00 €	0,00 €
Enterprise Architect 9	Modelado de casos de uso	1	0,00 € *	0,00 €
Sistema operativo	Windows / Linux	2	0,00 €	0,00 €
TOTAL				1.490,00 €

4.2. Presupuesto económico

El presente proyecto ha sido desarrollado íntegramente durante el periodo de prácticas profesionales de los alumnos en la empresa Sercampo, como parte de la formación del ciclo formativo de Desarrollo de Aplicaciones Web. Gracias a este contexto, la empresa no ha tenido que realizar ninguna inversión económica directa para la obtención de la aplicación, ya que se han aprovechado los recursos materiales que ya poseía y el trabajo ha sido realizado por los alumnos en el marco de su formación práctica.

A continuación, se presenta el desglose del presupuesto, diferenciando entre los costes que habría supuesto el proyecto en un entorno profesional y el coste real asumido por la empresa.

<i>Concepto</i>	<i>Coste profesional estimado</i>
<i>Recursos humanos</i>	
Analista/desarrollador(2 personas, 3 meses)	12.000,00 €
<i>Recursos materiales</i>	
Hardware(equipos proporcionados por la empresa)	1.490,00 €
Software (herramientas gratuitas)	0,00 €
<i>Otros gastos</i>	
TOTAL	13.490,00 €

5. Recursos humanos

Para la ejecución del proyecto se ha contado con un equipo humano reducido pero polivalente, compuesto por los dos desarrolladores en formación, complementado por figuras de supervisión y apoyo técnico dentro de la estructura de Sercampo. El desarrollo se ha llevado a cabo en el marco de las prácticas profesionales del ciclo formativo de Desarrollo de Aplicaciones Web, por tanto no se ha seguido una estructura clara en cuanto a roles de equipo ni definición de competencias.

5.1. Organización

La organización del trabajo durante el proyecto ha estado basada en un modelo colaborativo y flexible, sin roles fijos ni asignaciones excluyentes. El equipo ha estado formado por Laura Olmedilla García y Asier Martínez Muñoz, quienes han realizado todas las fases del desarrollo: análisis de requisitos, diseño de la base de datos, implementación del backend en PHP, desarrollo del frontend con Angular, integración

de la API REST, pruebas funcionales, depuración de incidencias y elaboración de la documentación técnica y de usuario.

Ambos miembros han trabajado de forma conjunta en cada módulo, compartiendo conocimientos y resolviendo de manera colaborativa los problemas técnicos surgidos durante el proceso. Esta situación ha permitido que ambos perfiles adquirieran competencias tanto en el lado del cliente como del lado del servidor, redundando en una mayor calidad del producto final y en una mejor preparación profesional.

Para la coordinación del trabajo se ha utilizado un tablero en Notion, tal como se ha descrito en el apartado 3.3 de esta memoria. En él se registraban las tareas pendientes, se asignaban responsabilidades de forma dinámica según la disponibilidad y se realizaba el seguimiento del progreso. Las reuniones de coordinación se han mantenido a diario de manera informal, dada la proximidad física en el puesto de trabajo, complementadas con una puesta en común más estructurada cada par de días para revisar los hitos alcanzados y planificar las iteraciones siguientes.

Además del equipo de desarrollo, se han contado con dos figuras de apoyo dentro de la empresa:

- Gustavo, quien ha actuado como supervisor puntual de nuestro trabajo. Sus funciones han incluido el seguimiento periódico del avance del proyecto, la validación de que los objetivos planteados se ajustaban a las necesidades reales de Sercampo y la provisión de directrices generales sobre la dirección a seguir.
- Ángel, como perfil senior de referencia, ha sido la persona a la que se ha consultado de forma puntual para resolver dudas técnicas específicas, especialmente en temas relacionados con el front, refactorización de código y la aportación de nuevos puntos de vista en problemas de estructura del código que se extendían más tiempo del necesario.

Ambos además han sido “árbitros” en decisiones relacionadas con la lógica detrás de la aplicación en los que, los desarrolladores principales, no llegábamos a un punto en común, por ser ambas opciones igual de válidas.

- Elena, es la persona en la empresa para la cual está diseñada esta aplicación, ya que será ella la persona que más la necesita y la que más uso le va a dar, acompañada de Paula, para aclarar qué necesidades reales hay detrás de las distintas consultas, que es más importante que sea accesible y que páginas hay que dedicarles más tiempo por ser la parte principal de su trabajo. Así como mejoras posibles que los desarrolladores debíamos encontrar cómo traspasar a nuestra aplicación.

Esta estructura organizativa, basada en la colaboración continua entre los dos desarrolladores, la supervisión intermitente pero cualificada, de un senior que al

mismo tiempo actúa de cliente y el apoyo experto bajo demanda, ha demostrado ser eficaz para completar el proyecto dentro del plazo establecido y con el nivel de calidad requerido.

5.2. Contratación

El presente proyecto se ha desarrollado íntegramente durante el período de prácticas profesionales de los alumnos en la empresa Sercampo, como parte de la formación del ciclo formativo de Desarrollo de Aplicaciones Web. En este contexto, no ha existido una relación laboral con contrato de trabajo en los términos del Estatuto de los Trabajadores, sino un acuerdo de formación práctica.

El período de ejecución del proyecto ha coincidido con la duración de las prácticas, que ha sido de tres meses, comprendidos entre el 3 de marzo y el 20 de mayo de 2026. Durante este tiempo, Laura Olmedilla García y Asier Martínez Muñoz han dedicado su jornada completa al desarrollo de la aplicación, bajo la supervisión de los tutores designados por la empresa y del centro educativo.

Si se valorase el proyecto en un entorno profesional ajeno al formativo, la contratación de dos desarrolladores junior durante tres meses habría supuesto un coste estimado de 12.000,00 €, como consideramos en el apartado 4.2, incluyendo la parte proporcional de pagas extraordinarias y cotizaciones a la Seguridad Social.

No obstante, gracias al acuerdo de prácticas, este coste ha sido 0€ para la empresa Sercampo, lo que ha permitido obtener una aplicación funcional y de calidad sin inversión directa en recursos humanos, aprovechando el trabajo realizado por los alumnos en el marco de su formación.

5.3. Prevención de riesgos laborales

El desarrollo del proyecto se ha llevado a cabo íntegramente en un entorno de oficina, realizando tareas propias del desarrollo de software (programación, diseño de bases de datos, documentación, reuniones de coordinación, etc.). Los riesgos laborales asociados son, por tanto, son los comunes a cualquier trabajo de oficina con uso intensivo de ordenadores. A continuación, se identifican dichos riesgos y se describen las medidas preventivas adoptadas.

Riesgos ergonómicos:

- Trabajo prolongado con pantallas de visualización de datos (monitores, ratón, teclado).

- Posturas forzadas derivadas de permanecer sentado durante largos períodos en la misma posición.
- Fatiga visual por enfoque continuo en pantallas de alta resolución.

Medidas preventivas adoptadas:

- Utilización de sillas ergonómicas con ajuste de altura y respaldo lumbar, proporcionadas por la empresa.
- Colocación de las pantallas a una distancia y altura adecuadas, de modo que el brazo quede extendido y el borde superior del monitor se sitúe a la altura de los ojos.
- Realización de pausas activas cada 1 o 2 horas, con ejercicios de estiramiento y cambios posturales breves (levantarse, caminar, estirar cuello y hombros).
- Alternancia de tareas entre programación, documentación y reuniones para evitar la monotonía postural y visual.
- Iluminación adecuada del puesto de trabajo, combinando luz natural y artificial para evitar reflejos en las pantallas.

Riesgos psicosociales:

- Carga mental y estrés asociados a los plazos de entrega, la resolución de errores complejos y la coordinación en equipo.
- Fatiga por atención continua durante largas sesiones de depuración de código o análisis de requisitos.

Medidas preventivas adoptadas:

- Planificación realista de tareas mediante el tablero de Notion, con asignación de prioridades y tiempos estimados, tal como se ha descrito en el apartado 3.3.
- Revisión diaria del progreso para detectar desviaciones a tiempo y reajustar las cargas de trabajo.

- Fomento de un ambiente de colaboración y apoyo mutuo entre los dos desarrolladores, evitando la competitividad interna.
- Comunicación fluida con los supervisores (Gustavo y Ángel).
- Establecimiento de un horario de trabajo continuo con interrupciones planificadas (descanso para almorzar, pausas breves, tiempo para el café).

Riesgos de seguridad (accidentes) en oficina:

- Caídas al mismo nivel por cables mal gestionados o cajas en el camino, además del mal ajuste del sensor de movimiento que enciende la luz.
- Golpes o cortes con mobiliario o material de oficina.

Medidas preventivas adoptadas:

- Mantenimiento del puesto de trabajo ordenado, sin objetos que puedan caer o causar daño al material suministrado.
- Conocimiento de las salidas de emergencia y extintores del edificio, conforme a la formación de acogida proporcionada por la empresa.

En conclusión, aunque el desarrollo de software no presenta riesgos laborales de alta gravedad, se han aplicado las buenas prácticas de prevención en entornos de oficina para garantizar la salud y el bienestar de los desarrolladores durante todo el período de ejecución del proyecto, contribuyendo así a un entorno de trabajo seguro y productivo.

6. Viabilidad técnica

El análisis de la viabilidad técnica tiene como objetivo determinar si el proyecto propuesto es realizable desde el punto de vista tecnológico, considerando los recursos disponibles, los conocimientos del equipo desarrollador, las herramientas empleadas y la infraestructura necesaria para su puesta en funcionamiento.

6.1. Estudio de viabilidad técnica

El estudio de viabilidad técnica del proyecto SERCAMPONET ha partido de la premisa de desarrollar una aplicación web completa, funcional y escalable para la gestión integral de una empresa de recogida y reciclaje de aceite. Para ello, se ha evaluado

cada uno de los componentes tecnológicos necesarios, así como las capacidades del equipo para abordarlos.

El stack tecnológico seleccionado ha sido el siguiente:

- Frontend: Angular 21 como framework para el desarrollo de Single Page Application (SPA).
- Backend: PHP, implementando una API REST con arquitectura MVC.
- Base de datos: MySQL, gestionada mediante phpMyAdmin y consultas preparadas con PDO.
- Estilos y maquetación: Bootstrap 5, CSS personalizado y Bootstrap Icons.
- Entorno de desarrollo local: XAMPP (Apache, MySQL, PHP).
- Control de versiones: Git y GitHub.
- Gestión de incidencias, tareas y organización: Notion.

La elección de estas tecnologías se ha basado en criterios de robustez, madurez, documentación disponible, comunidad activa y, especialmente, en su adecuación a los conocimientos adquiridos durante el ciclo formativo de Desarrollo de Aplicaciones Web así como interés en especializarse en ellos y completar la formación.

7. Viabilidad económica-financiera

En este apartado se analiza la viabilidad económica del proyecto, considerando tanto los gastos e inversiones necesarios para su desarrollo como las posibles fuentes de financiación y la rentabilidad esperada en caso de que la empresa Sercampo decida implantar la solución de forma definitiva.

7.1. Inversiones y gastos

Para la realización de este proyecto se han considerado los siguientes conceptos de inversión y gasto, diferenciando entre aquellos asociados al desarrollo y los necesarios para el posterior despliegue y mantenimiento en la empresa.

Concepto	Coste (€)	Observaciones
Recursos humanos	12.000 €	Estimación de 300 horas totales a 40 €/hora
Hardware (equipos de desarrollo)	1.200 €	Uso de ordenadores personales (amortización del 20% del valor de equipo de 1200 € durante 4 meses)
Software y licencias	0 €	Uso de tecnologías open source (Angular, PHP, MySQL, Bootstrap, XAMPP, Git)
Servidor de hosting (primer año)	240 €	Hosting básico con soporte para PHP y MySQL (20 €/mes) – para entorno de producción real
Dominio	15 €	Registro de dominio anual (.com o .es)
Capacitación y soporte inicial	500 €	Formación a los empleados de Sercampo y acompañamiento durante las primeras semanas

Gastos imprevistos (10%)	1.396 €	Para cubrir posibles necesidades adicionales
TOTAL ESTIMADO	15.351 €	

7.2. Financiación

Dado que el proyecto se ha desarrollado en el marco académico del ciclo formativo de Desarrollo de Aplicaciones Web, la fase de desarrollo no ha requerido financiación externa, ya que los recursos humanos han sido aportados por los alumnos y los recursos materiales (ordenadores, software) han sido proporcionados por la propia empresa.

En caso de que Sercampo desee implantar la aplicación de forma definitiva, se plantean las siguientes fuentes de financiación:

- Autofinanciación empresarial: La empresa puede asumir los costes de despliegue, dominio y formación con sus propios recursos, dado que el importe estimado (inferior a 1.000 € para el primer año sin costes de desarrollo) es asumible para una pyme del sector.
- Subvenciones y ayudas: Sercampo podría optar a programas de digitalización para pequeñas y medianas empresas, como las ayudas del programa Kit Digital (Gobierno de España), que financian parcial o totalmente la implantación de soluciones tecnológicas.
- Financiación bancaria: En caso de necesitar aplazar el pago, el importe reducido permitiría acceder a microcréditos o líneas de financiación a corto plazo con entidades colaboradoras.

7.3. Viabilidad económico-financiera

El proyecto es económicamente viable tanto en el escenario académico como en una hipotética implantación real. La inversión necesaria es reducida (especialmente si se aprovecha el desarrollo ya realizado), los costes de mantenimiento son bajos y los beneficios, tanto cuantitativos como cualitativos, superan ampliamente los gastos estimados. Se recomienda a Sercampo proceder al despliegue definitivo de la aplicación.

8. Conclusión

Este proyecto final surgió como una necesidad real de dar solución a un problema en la empresa Sercampo: digitalizar de forma adecuada un sistema de recogida de datos obsoleto y sin automatizar. Para ello, tuvimos diferentes reuniones con el equipo de Sercampo y las personas implicadas en su utilización y para las que está pensada esta aplicación. La finalidad de esta memoria ha sido documentar todo el proceso de desarrollo de la aplicación y poder así implementar el sistema y evaluar sus objetivos.

Una vez finalizado el desarrollo, podemos confirmar que se ha cumplido con los objetivos marcados al inicio del proyecto. Se ha diseñado e implementado una aplicación web funcional, basada en una arquitectura cliente-servidor, que permite la gestión integral de clientes, contenedores, rutas, recogidas y usuarios. La base de datos relacional desarrollada en MySQL garantiza la integridad, coherencia y escalabilidad de los datos, mientras que la interfaz construida con Angular y Bootstrap ofrece una experiencia de usuario intuitiva, accesible y adaptada a los diferentes roles del sistema (administrador, usuario estándar y conductor).

A lo largo del proceso, se han aplicado buenas prácticas de desarrollo, como la normalización de la base de datos, el uso del patrón MVC en el servidor, la validación de formularios tanto en cliente como en servidor, y la implementación de un sistema de autenticación seguro. Además, la documentación detallada, la guía de estilos y el manual de usuario facilitarán el mantenimiento y la adopción de la herramienta en el entorno real de la empresa.

Durante el proceso de desarrollo hemos tenido muchos aprendizajes, destacamos la importancia de una fase de análisis rigurosa, la necesidad de mantener una comunicación constante con los usuarios finales y entre nosotros para ajustar los requisitos, y la integración de tecnologías como Angular para el frontend y PHP con PDO para el backend.

También hemos identificado áreas de mejora y funcionalidades que podrían incorporarse en futuras versiones, como la geolocalización avanzada de contenedores, la generación automática de rutas optimizadas, o la implementación de un sistema de alertas y notificaciones.

En conclusión, el proyecto no solo ha permitido adquirir y afianzar conocimientos técnicos propios del título de Técnico Superior en Desarrollo de Aplicaciones Web, sino que ha demostrado que es posible transformar un proceso manual y obsoleto en una herramienta digital eficiente, trazable y adaptada a las necesidades reales de una empresa del sector del reciclaje y la logística. La aplicación desarrollada supone una base sólida para la transformación digital de Sercampo y abre la puerta a futuras ampliaciones que consoliden su crecimiento y mejora continua.

Bibliografía/Webgrafía

Documentación de sitios oficiales:

[Documentación Oficial Angular](#)
[Documentación Oficial Bootstrap](#)
[Documentación Oficial PHP](#)
[Documentación Oficial GitHub](#)
[Documentación Oficial MySql](#)
[Documentación Oficial Postman](#)

Recursos:

[Iconos Bootstrap](#)
[Snippets para sombras en css](#)
[Excalidraw](#)
[Draw.io](#)
[Pinterest](#)
[Figma](#)

Otros:

[w3school](#)
[Tutorial Angular](#)
[Curso Angular Pildoras Informáticas](#)
[Angular Signals](#)
[Angular Services](#)
[Deepseek AI](#)

ANEXOS

https://github.com/AsierMartinez2000/Proyecto_Sercampo_BaseDatos/tree/develop/Anexos